ใบอนุมัติโครงงานปริญญานิพนธ์
สาขาวิชาวิทยาการคอมพิวเตอร์ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

ชื่อปริญญานิพนธ์ การจำลองระบบหุ่นยนต์รับ-ส่งสิ่งของในโรงพยาบาลแผนกผู้ติดเชื้อโควิด-19
Simulation of a Delivery Robots System in the Hospitals Department Infected by COVID-19

ผู้จัดทำ นายนนทกร จั่นเชย
นางสาวศศิธร ทิพพรหม

ปีการศึกษา 2565

โครงงานปริญญานิพนธ์นี้ได้รับการอนุมัติให้เป็นส่วนหนึ่งของการศึกษาตาม
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
ภาควิชาคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

..... ประธานกรรมการสอบ
(อาจารย์เสฐลัทธ์ รอดเหตุภัย)

..... กรรมการสอบ
(อาจารย์ ดร.วัสรา รอดเหตุภัย)

..... อาจารย์ที่ปรึกษาและกรรมการสอบ
(ผู้ช่วยศาสตราจารย์ ดร. กฤษณะ สีพนมวัน)

การจำลองระบบหุ่นยนต์รับ-ส่งสิ่งของในโรงพยาบาลแผนกผู้ติดเชื้อโควิด19

Simulation of a Delivery Robots System in the Hospitals Department Infected by COVID-19

นนทกร จั่นเชย

ศศิธร ทิพพรหม

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาในหลักสูตรวิทยาศาสตรบัณฑิต

สาขาวิชาวิทยาการคอมพิวเตอร์ ภาควิชาคอมพิวเตอร์

คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

ปีการศึกษา 2565

ชื่อปริญญานิพนธ์ การจำลองระบบหุ่นยนต์รับ-ส่งสิ่งของในโรงพยาบาลแผนกผู้ติดเชื้อโควิด-19

Simulation of a Delivery Robots System in the Hospitals Department Infected by COVID-19

ผู้จัดทำ นายนนทกร จั่นเชย, นางสาวศศิธร ทิพพรหม

อาจารย์ที่ปรึกษา ผู้ช่วยศาสตราจารย์ ดร. กฤษณะ สีพนมวัน

ชื่อปริญญา วิทยาศาสตรบัณฑิต (วิทยาการคอมพิวเตอร์)

ปีการศึกษา 2565

บทคัดย่อ

ปริญญานิพนธ์เรื่องการจำลองหุ่นยนต์-รับส่งสิ่งของในโรงพยาบาลแผนกผู้ติดเชื้อโควิด-19 โดยใช้ ROS และ Gazebo เพื่อใช้ในการศึกษาการนำเทคโนโลยีหุ่นยนต์มาใช้งานในโรงพยาบาล เพื่อลดจำนวนการติดเชื้อโควิด-19 โดยระบบจะแบ่งการทำงานออกเป็น 3 ส่วนได้แก่ 1. ระบบติดต่อกับผู้ใช้งาน โดยระบบนี้จะทำหน้าที่ในการช่วยให้ผู้ใช้งานสามารถสั่งการหุ่นยนต์ได้ โดยผู้ใช้งานจะมีทั้งหมด 3 ประเภทคือ หมอ พยาบาล และ ผู้ดูแลระบบ ในส่วนของหมอและพยาบาลนั้น มีความสามารถในการใช้เว็ปแอปพลิเคชันเพื่อสั่งงานหุ่นยนต์ทำงานตามโหมดต่าง ๆ ผู้ดูแลระบบจะมีความสามารถในการใช้เว็ปแอปพลิเคชันเพื่อเข้าถึงการตั้งค่าต่าง ๆ ของหุ่นยนต์ พิกัดห้อง และผู้มีสิทธิเข้าใช้งานเว็ปแอปพลิเคชัน 2. ระบบฐานข้อมูล ทำหน้าที่ในการเก็บประวัติการทำงานของหุ่นยนต์ ข้อมูลต่าง ๆ ที่หุ่นยนต์จำเป็นต้องใช้ในการทำงาน โดยใช้ MySQL 3. ระบบการทำงานของหุ่นยนต์ จะทำหน้าที่รับคำสั่งจากผู้ใช้งานและทำการวางแผนการเคลื่อนที่อัตโนมัติเพื่อให้หุ่นยนต์สามารถทำงานได้ตามที่ได้รับมอบหมายโดยใช้การทำ Map Server และ Move Base ซึ่งปริญญานิพนธ์นี้จะเป็นการจำลองการใช้หุ่นยนต์ในโรงพยาบาล เพื่อเป็นแนวทางในการลดจำนวนการติดเชื้อโควิด-19 และโรคระบาดที่อาจเกิดขึ้นในอนาคตต่อไป

คำสำคัญ : การจำลอง, หุ่นยนต์ส่งของ, ระบบปฏิบัติการหุ่นยนต์, โปรแกรม Gazebo

Keyword : Simulation, Delivery Robots, Robot Operating System (ROS), Gazebo

กิตติกรรมประกาศ

ปริญญานิพนธ์เรื่องการจำลองหุ่นยนต์รับ-ส่งสิ่งของในโรงพยาบาลแผนกผู้ติดเชื้อโควิด19 สามารถสำเร็จลุล่วงไปได้ด้วยดีนั้น เนื่องมาจากได้รับความรู้ จากอาจารย์ที่ปรึกษาโครงงาน ผู้ช่วยศาสตราจารย์ ดร.กฤษณะ สีพนมวัน กรรมการโครงงาน อาจารย์เสฐลัทธ์ รอดเหตุภัย และ อาจารย์ ดร.วัสรา รอดเหตุภัย ผู้ให้คำปรึกษาและให้การแนะนำในการศึกษาค้นคว้าขั้นตอนและวิธีการให้สำเร็จลุล่วงได้ด้วยดีขอกราบขอบพระคุณคณาจารย์ มหาวิทยาลัยศิลปากรคณะวิทยาศาสตร์และภาควิชาคอมพิวเตอร์ที่ให้วิชาความรู้และสถานที่ในการจัดทำโครงงานปริญญานิพนธ์นี้

สุดท้ายนี้ขอกราบขอบพระคุณ คุณพ่อและคุณแม่ รวมถึงญาติทุกๆ คนที่คอยให้การสนับสนุนเสมอมา และขอขอบคุณผู้ตอบปัญหาจากเว็บไซต์ต่าง ๆ ที่ทำให้ผู้จัดทำสามารถแก้ไขปัญหาที่พบเจอได้ ผู้จัดทำจึงขอกราบขอบพระคุณเป็นอย่างสูง ณ โอกาสนี้

นนทกร จั่นเชย

ศศิธร ทิพพรหม

สารบัญ

หน้า

บทคัดย่อ ..... ก
กิตติกรรมประกาศ ..... ข
สารบัญ ..... ค
สารบัญตาราง ..... ฉ
สารบัญรูป ..... ช
บทที่ 1 บทนำ 1
1.1 ที่มาและความสำคัญของปัญหา 1
1.2 วัตถุประสงค์ 1
1.3 ลักษณะและขอบเขต 1
1.4 อุปกรณ์และเครื่องมือที่ใช้ 2
1.4.1 ฮาร์ดแวร์ 2
1.4.2 ซอฟต์แวร์ 2
1.5 ผลที่คาดว่าจะได้รับ 2
1.6 ภาระหน้าที่ความรับผิดชอบ 2
บทที่ 2 ผลงานที่เกี่ยวข้อง 4
2.1 CARVER หุ่นยนต์เคลื่อนที่อัตโนมัติ ขนส่งอาหาร ยา เวชภัณฑ์ในโรงพยาบาล [1] 4
2.2 A Study on Global Path Planners Algorithms for the Simulated TurtleBot 3 Robot in ROS [2] 5
2.2.1 A* Algorithm [3] 5
2.2.2 Dijkstra Algorithm [4] 6
2.3 Mobile Robot Simulation and Navigation in ROS and Gazebo [5] 7
2.3.1 การใช้ ROS และ GAZEBO 7
บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง 8
3.1 Robot Operating System (ROS) [6] 8
3.2 Gazebo [7] 9
3.3 Rviz [8] 10
3.4 MiR 100 [9] 11
3.5 Autonomous Navigation [10] 11
3.6 Simultaneous Localization and Mapping (SLAM) [11] 12
3.7 Adaptive Monte Carlo Localization (AMCL) [12] 12
3.8 Sensor Robot [13] 12
3.9 MySQL [14] 12
3.10 Web Video Server [15] 13
3.11 Rosbridge_suite [16] 13
3.12 Roslibjs [17] 13

3.13 Ros2djs [18] 14
3.14 EvenEmitter2 [19] 15
3.15 EaseUs [20] 15
บทที่ 4 ขั้นตอนและแผนการดำเนินงาน 16
4.1 ขั้นตอนการดำเนินงาน 16
4.1.1 การรวบรวมข้อมูลและกำหนดขอบเขต 16
4.1.2 การศึกษาและเรียนรู้ทฤษฎีที่เกี่ยวข้อง 16
4.1.3 การวิเคราะห์และออกแบบระบบ 16
4.1.4 การพัฒนาระบบ 16
4.1.5 การทดสอบระบบและปรับปรุง 16
4.1.6 การจัดทำเอกสารประกอบโครงงาน 16
4.2 แผนการดำเนินงาน 17
บทที่ 5 การวิเคราะห์และออกแบบระบบ 18
5.1 โครงสร้างของระบบ 18
5.2 หลักการทำงานของระบบ 19
5.2.1 การทำงานของหุ่นยนต์ 19
5.2.2 ระบบติดต่อกับผู้ใช้งาน 20
5.2.3 ระบบฐานข้อมูล 24
5.3 โครงสร้างการทำงานของหุ่นยนต์ 25
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ 26
6.1 ผลการดำเนินงาน 26
6.1.1 การใช้งานหุ่นยนต์ของหมอและพยาบาล 26
6.1.2 การทำงานของหุ่นยนต์ 28
6.2 ข้อจำกัด 29
6.3 ข้อเสนอแนะ 29
บรรณานุกรม 30
ภาคผนวก ก การติดตั้งโปรแกรมในการทำงาน 32
ก.1 การติดตั้ง ROS Noetic 32
ก.1.1 การตั้งค่า source. List 32
ก.1.2 การตั้งค่าคีย์ 32
ก.1.3 การติดตั้ง ROS Noetic 32
ก.1.4 การตั้งค่า Environment 32
ก.2 การติดตั้ง Web Video Server 32
ก.2.1 การติดตั้ง Web Video Server 32
ก.3 การติดตั้ง XAMPP 33
ก.3.1 การดาวน์โหลด XAMPP 33
ก.4 การติดตั้ง Flask 34

ก.4.1 การตรวจสอบการUpdateก่อนติดตั้ง 34
ก.4.2 การติดตั้ง pip3 และเครื่องมือ Python อื่น ๆ 34
ก.4.3 การติดตั้งEnvironment 35
ก.4.4 การสร้างไดเร็กทอรี flask 35
ก.4.5 การสร้าง Environment โดยใช้ Python 35
ก.4.6 การติดตั้ง เว็บเฟรมเวิร์ก flask โดยใช้ pip 35
ภาคผนวก ข คู่มือการใช้งาน 36

สารบัญตาราง

หน้า

ตารางที่ 4.1 แผนการดำเนินงาน..... 17

สารบัญรูป

หน้า

รูปที่ 2.1 CARVER หุ่นยนต์เคลื่อนที่อัตโนมัติ 4
รูปที่ 3.1 ตัวอย่างการทำงานของ ROS 8
รูปที่ 3.2 โปรแกรม Gazebo 9
รูปที่ 3.3 การจำลองหุ่นยนต์หลักใน Gazebo 9
รูปที่ 3.4 ฮาร์ดแวร์หลักของหุ่นยนต์ 10
รูปที่ 3.5 TurtleBot3 บนโปรแกรม Rviz 10
รูปที่ 3.6 โมเดลของ MiR 100 11
รูปที่ 3.7 องค์ประกอบของ Autonomous Navigation 11
รูปที่ 3.8 การทำงานของ Web_Video_Server 13
รูปที่ 3.9 ตัวอย่างโค้ด Roslibjs 14
รูปที่ 3.10 ตัวอย่างโค้ด การนำภาพจาก /map ขึ้นเว็บด้วย ros2djs 14
รูปที่ 3.11 ตัวอย่างโค้ด การทำสัญลักษณ์ติดตามหุ่นยนต์บนแผนที่เว็บ 15
รูปที่ 5.1 โครงสร้างการทำงานของระบบ 18
รูปที่ 5.2 การทำงานระหว่างหมอและพยาบาล กับหุ่นยนต์ 20
รูปที่ 5.3 หน้า UI เข้าสู่ระบบการสั่งการหุ่นยนต์ 20
รูปที่ 5.4 หน้า UI ของหมอและพยาบาลในโหมดส่งของ 20
รูปที่ 5.5 หน้า UI โหมดส่งขอซึ่งแสดงปุ่มลบ 21
รูปที่ 5.6 หน้า UI โหมดติดต่อผู้ป่วย 21
รูปที่ 5.7 หน้า UI โหมดติดต่อผู้ป่วยหลังกด สั่งทำงานหุ่นยนต์ 22
รูปที่ 5.8 หน้า UI โหมดส่งหุ่นยนต์ 23
รูปที่ 5.9 ภาพตัวอย่างสถานะหุ่นยนต์ พร้อมใช้งานและไม่พร้อมใช้งาน 23
รูปที่ 5.10 การตอบสนองระหว่างผู้ป่วยกับหุ่นยนต์ 23
รูปที่ 5.11 ตารางข้อมูลที่ใช้ในการทำงานด้วย MySQL 24
รูปที่ 5.12 หน้า UI ของผู้พัฒนาระบบ 25
รูปที่ 5.13 โครงสร้างการทำงานของหุ่นยนต์ 25
รูปที่ 6.1 หุ่นยนต์ขณะปิดกระจกชั้นวางของ 26
รูปที่ 6.2 หุ่นยนต์เปิดชั้นวางของเพื่อให้ผู้รับหยิบของ 26
รูปที่ 6.3 การติดต่อผู้ป่วยของหมอและพยาบาล 27
รูปที่ 6.4 การติดตามตำแหน่งหุ่นยนต์ 27
รูปที่ 6.5 เหตุการณ์เมื่อมีสิ่งกีดขวาง ขวางเส้นทาง 28
รูปที่ 6.6 การวางแผนเส้นทางใหม่ เพื่อหลบสิ่งกีดขวาง 28
รูปผนวกที่ ก.1 รูปแสดงดาวน์โหลด XAMPP 33
รูปผนวกที่ ก.2 รูปแสดงหน้าจอติดตั้งโปรแกรม XAMPP 34
รูปผนวกที่ ก.3 รูปแสดงหน้าจอติดตั้งโปรแกรม XAMPP 34

รูปผนวกที่ ข.1 รูปแสดงหน้าเข้าสู่ระบบ 36
รูปผนวกที่ ข.2 รูปแสดงหน้าเข้าสู่ระบบ มีการแจ้งเตือนข้อผิดพลาดขึ้น 36
รูปผนวกที่ ข.3 รูปแสดงหมวดการทำงานของหมอโหมดส่งของ 36
รูปผนวกที่ ข.4 รูปแสดงโหมดส่งของ แสดงคิวการทำงานที่เพิ่มไป และสามารถกดลบได้ 37
รูปผนวกที่ ข.5 รูปแสดง Feedback ให้เห็นว่าสั่งสำเร็จแล้ว 37
รูปผนวกที่ ข.6 รูปแสดงโหมดติดต่อผู้ป่วย 37
รูปผนวกที่ ข.7 รูปแสดงโหมดส่งหุ่นยนต์ 38
รูปผนวกที่ ข.8 รูปแสดงฟังก์ชันประวัติการทำงาน 38
รูปผนวกที่ ข.9 รูปแสดงฟังก์ชันการเพิ่มข้อมูลผู้ป่วย 38
รูปผนวกที่ ข.10 รูปแสดงฟังก์ชันประวัติผู้ป่วย 39
รูปผนวกที่ ข.11 รูปแสดงฟังก์ชันการตั้งค่าผู้ใช้งาน 39
รูปผนวกที่ ข.12 รูปแสดงฟังก์ชันการตั้งค่าพิกัดปลายทาง 39
รูปผนวกที่ ข.13 รูปแสดงฟังก์ชันการตั้งค่าหุ่นยนต์ 40

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

ในปัจจุบันการประยุกต์ใช้งานหุ่นยนต์ เข้ามามีบทบาทในชีวิตประจำวันและจะมีความสำคัญมากยิ่งขึ้นในอนาคต ไม่เพียงแต่ในภาคอุตสาหกรรม ซึ่งการเข้ามาของหุ่นยนต์ทำให้ต้นทุนการผลิตลดลงและประสิทธิภาพการทำงานเพิ่มขึ้น ยังมีการพัฒนาเทคโนโลยีด้านต่าง ๆ สำหรับยกระดับคุณภาพชีวิต ไม่ว่าจะเป็น HealthTech เทคโนโลยีเพื่อดูแลเรื่องสุขภาพ, FoodTech เทคโนโลยีเกี่ยวข้องกับอาหารและเกษตรกรรม, การอำนวยความสะดวกในที่อยู่อาศัย รวมถึงความปลอดภัย, การกู้ภัย, การรักษาทางการแพทย์ แม้กระทั่งความบันเทิงและสันทนาการ เป็นต้น

เนื่องจากในปัจจุบันมีโรคระบาดที่ชื่อว่าโควิด-19 เป็นโรคที่มีความเสี่ยงสูงที่จะติดต่อได้ง่าย โดยเฉพาะบุคลากรทางการแพทย์ที่อยู่ใกล้กับผู้ป่วยอยู่เป็นประจำ ทำให้มีความเสี่ยงมากเป็นพิเศษ เพราะต้องทำการรักษาคนไข้ที่ได้รับเชื้อโควิด-19 จึงต้องระวังเป็นอย่างมาก และ ในโรงพยาบาลหลายๆ โรงพยาบาลมีผู้มาใช้บริการเป็นจำนวนมากในทุกๆ วัน และในบางครั้งอาจเกิดปัญหาบุคคลากรทางการแพทย์มีไม่เพียงพอกับผู้มาใช้บริการ เนื่องจากบุคคลากรมีจำนวนน้อย ทำให้ทุกภาคส่วนมีงานแน่นตลอดเวลา หรือ อาจเกิดเหตุการณ์ที่บุคลากรเกิดการติดเชื้อโควิด-19 จากการเข้าใกล้ผู้ป่วย ทำให้ไม่สามารถทำงานได้และทำให้บุคคลากรยิ่งน้อยลงไปกว่าเดิม เพื่อที่จะแก้ปัญหาเหล่านี้ผู้จัดทำจึงได้ประยุกต์การใช้งานของหุ่นยนต์มาเพื่อช่วยลดภาระในการทำงานบางอย่างออกไป

ทางผู้จัดทำจึงได้ประยุกต์ใช้หุ่นยนต์ในการเข้ามาแก้ปัญหาที่กล่าวไว้ข้างต้น ในกรณีศึกษาในโรงพยาบาลแผนกของผู้ที่ถูกกักตัวจากโรคระบาดโควิด-19 เป็นการจำลองการสร้างหุ่นยนต์ที่เข้ามาช่วยทำหน้าที่แทนงานที่มีความเสี่ยงต่อการติดเชื้อโควิด-19 เช่น การส่งอาหารและเครื่องดื่ม ให้กับผู้ป่วยที่ติดเชื้อโควิด-19 การดูแลและติดตามอาการของผู้ป่วยแทนหมอและพยาบาล จะสามารถลดภาระงาน ลดการติดของเชื้อโควิด-19 อีกทั้งยังประหยัดงบในการจัดหาอุปกรณ์ป้องกันเชื้อโควิด19 อย่างเช่นชุด PPE และหน้ากากอนามัยได้ เนื่องจากในอนาคตอาจจะเกิดโรคระบาดขึ้นได้อีกการประยุกต์ใช้หุ่นยนต์กับโรงพยาบาลจึงเป็นเรื่องที่ดีสำหรับโรงพยาบาลในอนาคตที่จะเกิดโรคระบาดใหม่ๆ ขึ้น

1.2 วัตถุประสงค์

เพื่อพัฒนาระบบการบริหารจัดการ การใช้งานหุ่นยนต์และการบริการทางแพทย์ในโรงพยาบาล

1.3 ลักษณะและขอบเขต

โครงงานวิจัยนี้เป็นการจำลองการทำงานของหุ่นยนต์ที่ทำหน้าที่แทนบุคลากรทางการแพทย์ โดยทำหน้าที่ส่งเอกสาร อาหารเครื่องดื่ม และยาทั่วไป โดยการใช้ ROS และใช้ Gazebo ในการจำลองแผนที่ในโรงพยาบาลแผนกของผู้ที่ถูกกักตัวจากโรคระบาดโควิด-19 การทำงานหลักๆ ของหุ่นยนต์มีดังนี้

1) หุ่นยนต์จำลองสามารถเคลื่อนที่จากต้นทางไปยังปลายทางได้โดยอัตโนมัติ
2) หุ่นยนต์จำลองมีการเก็บข้อมูลการทำงานของหุ่นยนต์
3) หุ่นยนต์จำลองสามารถช่วยสื่อสารระยะไกลระหว่างบุคลากรทางการแพทย์และผู้ป่วย

โดยผู้ใช้งานจะแบ่งออกเป็น 3 ประเภทดังนี้

ประเภทที่ 1 ผู้ดูและระบบที่ทำหน้าที่ดูแล เพิ่ม ลบ แก้ไขข้อมูลขระบบของหุ่นยนต์จำลอง และประวัติการทำงานของหุ่นยนต์จำลอง

ประเภทที่ 2 บุคลากรทางการแพทย์ ที่ทำหน้าที่เรียกใช้บริการรับ-ส่งสิ่งของกับหุ่นยนต์จำลอง

ประเภทที่ 3 ผู้ป่วยที่ติดต่อสื่อสารกับแพทย์ทางจอภาพของหุ่นยนต์จำลอง

1.4 อุปกรณ์และเครื่องมือที่ใช้

1.4.1 ฮาร์ดแวร์

1) System Model : ASUS TUF Gaming A15 FA506ICB_FA506ICB
   Processor : AMD Ryzen 7 4800H with Radeon Graphics 2901 Mhz
   Graphic Card : NVIDIA® GeForce RTX™ 3050, 4GB GDDR6
   RAM : 8 GB
2) System Model : Notebook Acer Nitro AN515-52
   Processor : Intel (R) Core (TM) i5-8300H CPU @ 2.30GHz 2.30 GHz
   Graphic Card : NVIDIA® GeForce GTX 1050 Ti
   RAM : 8 GB

1.4.2 ซอฟต์แวร์

- Ubuntu 20.04.05 LTS
- ROS Noetic
- Gazebo
- Visual Studio Code
- HTML, CSS, JavaScript, Python, Flask

1.5 ผลที่คาดว่าจะได้รับ

1) ได้แบบจำลองเกี่ยวกับการทำงานของหุ่นยนต์อัตโนมัติและสามารถนำไปใช้งานได้
2) ได้แนวทางในการช่วยลดปัญหาการติดเชื้อโควิด-19 จากการใกล้ชิดกับผู้ป่วย

1.6 ภาระหน้าที่ความรับผิดชอบ

1) นายนนทกร จั่นเชย
   <u>รับผิดชอบ</u> จัดการรายละเอียดพื้นฐานของหุ่นยนต์ ออกแบบส่วนประกอบหุ่นยนต์ ดูแลระบบในส่วนของการทำงานด้านบุคลากรทางการแพทย์และการทำงานในส่วนของคนไข้

2) นางสาวศศิธร ทิพพรหม

<u>รับผิดชอบ</u> ปรับปรุงออกแบบสร้างพื้นที่สำหรับหุ่นยนต์ ออกแบบการโต้ตอบระหว่างผู้ใช้กับหุ่นยนต์ จัดการในส่วน Database ดูแลในส่วนของผู้ดูแลระบบ

3) ความรับผิดชอบร่วม

<u>รับผิดชอบ</u> จัดทำรูปเล่มรวบรวมข้อมูลและวางแผน แก้ไขรับปรุงทุกส่วนของหุ่นยนต์ เขียน Script ของหุ่นยนต์

บทที่ 2 ผลงานที่เกี่ยวข้อง

ในบทนี้ผู้จัดทำได้ศึกษาเกี่ยวกับผลงานที่เกี่ยวข้อง จะกล่าวถึงลักษณะของงาน ข้อเด่นและข้อด้อย ของผลงานที่เกี่ยวข้องและมีความเหมาะสมในการพัฒนาโครงงานปริญญานิพนธ์นี้ ดังนี้

2.1 CARVER หุ่นยนต์เคลื่อนที่อัตโนมัติ ขนส่งอาหาร ยา เวชภัณฑ์ในโรงพยาบาล [1]

CARVER หุ่นยนต์เคลื่อนที่อัตโนมัติ ได้รับการพัฒนาคิดค้นเพื่อขนส่งอาหาร ยา เวชภัณฑ์ สำหรับผู้ป่วยในหอผู้ป่วยในโรงพยาบาล โดยหุ่นยนต์สามารถทำหน้าที่ขนส่งและบรรจุถาดได้มากถึง 20 ถาด/ครั้ง นอกจากนี้ยังมีฟังก์ชันฟอกอากาศและฆ่าเชื้อไวรัสผ่านอุปกรณ์ Hydroxyl Generator ตลอดการปฏิบัติงาน ทำให้ช่วยลดภาระการทำงานของบุคลากรทางการแพทย์และลดความเสี่ยงในการติดเชื้ออีกด้วย

โดย CARVER มีระบบที่เรียกว่า Autonomous Mobile Robot (AMR) เป็นหนึ่งในระบบอัตโนมัติที่ใช้กันอย่างแพร่หลายเพื่อปรับปรุงกระบวนการขนส่ง ในงานวิจัยนี้ AMRs ใช้สำหรับจัดส่งอาหารและเวชภัณฑ์ให้กับผู้ป่วยแต่ละรายโดยเฉพาะอย่างยิ่งในสถานการณ์แพร่ระบาดของโควิด-19 AMRs เป็นเครื่องมือสำคัญในการรักษาระยะห่างทางกายภาพระหว่างผู้ป่วยและเจ้าหน้าที่สาธารณสุขเพื่อป้องกันการติดเชื้อ ในงานวิจัยนี้ AMR ได้รับการติดตั้งโมดูล Internet of Things (IoT) ซึ่งสามารถเชื่อมต่อกับแพลตฟอร์ม IoT ทางฝั่งเซิร์ฟเวอร์ ส่งผลให้เจ้าหน้าที่สาธารณสุขสามารถตรวจสอบควบคุมหุ่นยนต์ได้อย่างมีประสิทธิภาพผ่านเว็บแอปพลิเคชัน โดยหุ่นยนต์ CARVER จะมีลักษณะดังรูปที่ 2.1

รูปที่ 2.1 CARVER หุ่นยนต์เคลื่อนที่อัตโนมัติ

Autonomous Mobile Robot (AMR) เป็นหนึ่งในโซลูชั่นสำหรับการปรับปรุงระบบโลจิสติกส์ AMR ใช้เพื่อขนส่งรายการระหว่างสถานที่เฉพาะโดยอัตโนมัติ มีการใช้กันอย่างแพร่หลายในอุตสาหกรรมมานานหลายทศวรรษและเริ่มนำไปใช้ในสภาพแวดล้อมการทำงานอื่น ๆ รวมถึงโรงพยาบาล ในช่วงสถานการณ์การแพร่ระบาดของไวรัส COVID-19 การนำ AMRs ไปใช้ในโรงพยาบาลไม่ได้เป็นเพียงการปรับปรุงประสิทธิภาพการทำงาน แต่ยังใช้สำหรับ วัตถุประสงค์ในการควบคุมการติดเชื้อ ควรลดการสัมผัสโดยตรงระหว่างเจ้าหน้าที่สาธารณสุขและผู้ป่วยเพื่อลดความเสี่ยงของการติดเชื้อ ดังนั้นจึงต้องพิจารณาวิธีใหม่ในการสื่อสารและการมีปฏิสัมพันธ์ระหว่างผู้ป่วยและเจ้าหน้าที่สาธารณสุข เพื่อหลีกเลี่ยงการสัมผัสโดยตรง หุ่นยนต์เคลื่อนที่จะถูกใช้โต้ตอบกับผู้ป่วย ในขณะที่เจ้าหน้าที่สาธารณสุขสามารถดำเนินการได้จากระยะไกลจากนอกพื้นที่ปนเปื้อน AMR ถูกนำมาใช้ในหลายกรณี เช่น การแพทย์ทางไกล การส่งสิ่งของ การฆ่าเชื้อโรคด้วยรังสียูวีซี และอื่น ๆ

ข้อดี

- ติดตั้งและใช้งานได้ง่าย
- ลดความเสี่ยงในการติดเชื้อของบุคลากรทางการแพทย์จากการใกล้ชิดกับผู้ป่วย
- สามารถบรรจุถาดขนส่งอาหาร ยาและเวชภัณฑ์ ได้มากถึง 20 ถาด/ครั้ง

ข้อด้อย

- GUI ของผู้ใช้ ไม่เหมาะสำหรับผู้ใช้ที่ไม่มีความชำนาญ
- เมื่อหุ่นยนต์เคลื่อนที่ไปยังตำแหน่งที่ไม่มีสัญญาณ Wi-Fi หุ่นยนต์ต้องสลับเครือข่ายไปยังแหล่งอื่น แต่ในบางครั้งอาจเกิดข้อผิดพลาดได้ จากการที่หุ่นยนต์ไม่สามารถกลับมาเชื่อมต่อสัญญาณ Wi-Fi ได้

2.2 A Study on Global Path Planners Algorithms for the Simulated TurtleBot 3 Robot in ROS [2]

บทความนี้เป็นการศึกษาอัลกอริทึม Global Path Planners โดยใช้หุ่นยนต์เคลื่อนที่ในกรอบ ROS (ระบบปฏิบัติการหุ่นยนต์) การทดลองดำเนินการในสภาพแวดล้อมที่แตกต่างกัน โดยมีจุดมุ่งหมายเพื่อใช้อัลกอริทึม Dijkstra และ A* ในการจำลอง การทดสอบดำเนินการกับหุ่นยนต์ TurtleBot 3 Burger ซึ่งทำงานโดยใช้ซอฟต์แวร์โอเพ่นซอร์ส ข้อมูลตำแหน่งและทิศทางจาก Sensor ของหุ่นยนต์ถูกเก็บไว้ หลังจากวิเคราะห์ผลลัพธ์ มีความเหนือกว่าเล็กน้อยของอัลกอริทึม Dijkstra จากไลบรารี ROS ซึ่งสัมพันธ์กับอัลกอริทึม A * ในสภาพแวดล้อมที่มีอุปสรรค

2.2.1 A* Algorithm [3]

A* คือการค้นหาเส้นทางและอัลกอริทึมการผ่านกราฟที่ใช้ในวิทยาการคอมพิวเตอร์ เนื่องจากมีความครบถ้วนสมบูรณ์และมีประสิทธิภาพในอุดมคติ โดยมองว่าเป็นส่วนขยายของอัลกอริทึมของ Dijkstra และปัจจุบันมีตัวแปรอื่น ๆ ที่มาจากอัลกอริทึมที่เสนอดั้งเดิม เป็นอัลกอริทึมการค้นหาที่กำหนดโดยกราฟิกถ่วงน้ำหนัก ซึ่งหมายความว่าการเริ่มต้นจากโหนดเริ่มต้นที่เฉพาะเจาะจงของแผนภูมิ มีเป้าหมายเพื่อค้นหาเส้นทางไปยังโหนดวัตถุประสงค์ที่มีต้นทุนต่ำที่สุด (ระยะทางที่สั้นที่สุด เวลาที่สั้นที่สุด ฯลฯ) ทำสิ่งนี้โดยการรักษาแผนผังของเส้นทางที่มาจากโหนดเริ่มต้นและขยายเส้นทางเหล่านี้ทีละขอบเขตจนกว่าจะถึงเกณฑ์ความสมบูรณ์

อัลกอริทึมนี้ใช้ฮิวริสติกเพื่อเพิ่มประสิทธิภาพการค้นหา สามารถกำหนดฟังก์ชันที่สามารถประเมินค่าใช้จ่ายจากโหนดหนึ่งไปยังเป้าหมายได้ในระดับหนึ่ง ข้อเสียในทางปฏิบัติที่สำคัญคือความซับซ้อนของพื้นที่ เนื่องจากพื้นที่ดังกล่าวเก็บโหนดทั้งหมดที่สร้างขึ้นในหน่วยความจำ A * ต้องตัดสินใจว่าเส้นทางใดจะพัฒนาในแต่ละการวนซ้ำของลูปหลัก สิ่งนี้ทำได้โดยพิจารณาจากต้นทุนของเส้นทาง และการประมาณการของต้นทุนที่จำเป็นในการขยายเส้นทางไปสู่เป้าหมาย A * ขยายเส้นทางที่มีการใช้ทรัพยากรน้อย

การใช้งาน A * มักจะใช้คิวลำดับความสำคัญเพื่อทำการเลือกโหนดค่าใช้จ่ายโดยประมาณขั้นต่ำซ้ำเพื่อขยาย ลำดับความสำคัญนี้เรียกว่า open list, open set หรือ fringe โหนดที่มีค่า a(x) ต่ำสุดจะถูกลบออกจากคิวในแต่ละขั้นตอนของอัลกอริทึม ค่า a และ c ที่อยู่ใกล้เคียงจะถูกอัพเดตและเพิ่มลงในคิว โหนดที่มีการวิเคราะห์เพื่อนบ้านทั้งหมดจะถูกจัดอยู่ในอีกรายการหนึ่ง ซึ่งเรียกว่ารายการปิด (closed list) อัลกอริทึมจะดำเนินต่อไปจนกว่าจะพบ เป้าหมายหรือรายการเปิด (open list) ว่างเปล่า วัตถุประสงค์คือค่าของต้นทุนเส้นทางที่สั้นที่สุด เนื่องจาก h ในวัตถุประสงค์เป็นศูนย์ ในฮิวริสติกที่อนุญาต

ซึ่งเป็นระยะทางที่สั้นที่สุดที่เป็นไปได้ระหว่างจุดสองจุด ถ้าฮิวริสติก h ตรงตามเงื่อนไขเพิ่มเติม $h(x) \leq l(x, y)+h(y)$ สำหรับแต่ละขอบเขต (x, y) ของกราฟ โดยที่ l หมายถึงความยาวของขอบเขตนั้น h จะเรียกว่าสอดคล้อง ด้วยฮิวริสติกที่สอดคล้องกัน A * รับประกันได้ว่าจะพบเส้นทางในอุดมคติโดยไม่ต้องประมวลผลโหนดใดๆ มากกว่าหนึ่งครั้ง ดังนั้น A * เทียบเท่ากับการรันอัลกอริทึม Dijkstra ด้วยต้นทุนที่ลดลงของสมการต่อไปนี้:

$$l'(x, y)=l(x, y)+h(y)-h(x)$$

2.2.2 Dijkstra Algorithm [4]

อัลกอริทึมของ Dijkstra คิดค้นขึ้นในปี 1956 โดย Edsger Dijkstra นักวิทยาศาสตร์คอมพิวเตอร์ชาวดัตช์ ตีพิมพ์ครั้งแรกในปี 1959 โดยมีวัตถุประสงค์เพื่อแก้ปัญหาเส้นทางขั้นต่ำ ปัญหาเส้นทางขั้นต่ำเป็นหนึ่งในปัญหาทั่วไปที่ได้รับการศึกษามากที่สุดในสาขาต่าง ๆ เช่น วิศวกรรมการขนส่ง การวิจัยเชิงปฏิบัติการ วิทยาการคอมพิวเตอร์ และปัญญาประดิษฐ์ ความแตกต่างที่สำคัญระหว่างอัลกอริทึม Dijkstra และ A* คือการขาดฟังก์ชันฮิวริสติกที่ช่วยอำนวยความสะดวกและลดจำนวนโหนดที่ขยาย เนื่องจากในทุกขั้นตอน อัลกอริทึม Dijkstra จะตรวจสอบโหนดที่อยู่ติดกันเพื่อดำเนินการประเมิน โดยไม่คำนึงถึงลำดับหรือการจัดลำดับความสำคัญ ของสาขาที่เกิดขึ้นในอัลกอริทึม A * เนื่องจากการใช้ฟังก์ชันฮิวริสติกที่กำหนดโดยปัญหา เมื่อเลือกจุดยอดเป็นรูทการค้นหาแล้ว อัลกอริทึมนี้จะคำนวณต้นทุนขั้นต่ำของจุดยอดนี้สำหรับจุดยอดอื่น ๆ ทั้งหมดของกราฟ นิพจน์กราฟหมายถึงคู่ (V, A) โดยที่ Vคือเซตของจุดยอด และ A คือเซตของคู่ของจุดยอด ซึ่งเรียกว่า เส้นขอบ สำหรับกรณีพิเศษที่ชุดของขอบเขตถูกกำหนดในรูปของคู่อันดับของจุดยอด กราฟจะได้รับชื่อว่ากราฟกำกับ ที่จุดยอด (Node) ที่กำหนดในกราฟ อัลกอริทึมจะค้นหาเส้นทางที่มีต้นทุนต่ำที่สุด (เส้นทางที่สั้นที่สุด) ระหว่างจุดยอดนั้นกับจุดยอดอื่น ๆ ทั้งหมด โดยใช้น้ำหนัก/ต้นทุนของขอบเขต ระบบนี้ยังสามารถใช้เพื่อค้นหาต้นทุนเส้นทางที่สั้นที่สุดจากจุดยอดเดียวไปยังจุดยอดปลายทางได้โดยการหยุดอัลกอริทึมเมื่อกำหนดเส้นทางที่สั้นที่สุดไปยังจุดยอดปลายทางแล้ว

ข้อดี

ในงานนี้ มีการจำลองสภาพแวดล้อมที่แตกต่างกันสองแบบใน Gazebo ซึ่งใช้ SLAM ร่วมกับไลบรารี Gmapping ใน ROS เพื่อสร้างแผนที่ของสภาพแวดล้อมจำลอง การทดสอบเกี่ยวกับการกระจัดของ TurtleBot 3 ประสบความสำเร็จ และทั้งอัลกอริทึมของ Dijkstra และ A* ให้ผลลัพธ์ที่ดีสำหรับการวางแผนวิถีการเคลื่อนที่ของหุ่นยนต์

ข้อด้อย

เนื่องจากสภาพแวดล้อมการทดลองที่มีทางเดินแคบ หุ่นยนต์อาจติดได้ง่าย หรือ คิดว่าเป็นไปไม่ได้ที่จะเคลื่อนที่ผ่านเส้นทางนั้น

ผลสรุปการเปรียบเทียบอัลกอริทึม

อัลกอริทึม Dijkstra ซึ่งมีอยู่ในไลบรารี ROS ให้ผลลัพธ์ที่ดีกว่าเล็กน้อยเมื่อเปรียบเทียบกับอัลกอริทึม A* ในสภาพแวดล้อมที่มีอุปสรรค

2.3 Mobile Robot Simulation and Navigation in ROS and Gazebo [5]

บทความนี้พูดถึง การจำลองกระบวนการพัฒนาหุ่นยนต์เคลื่อนที่ สำหรับการทดสอบส่วนประกอบซอฟต์แวร์ พฤติกรรมของหุ่นยนต์ และอัลกอริทึมการควบคุมในสภาพแวดล้อมโดยรอบที่แตกต่างกัน ในบทความนี้ แนะนำสภาพแวดล้อมจำลองสำหรับหุ่นยนต์เคลื่อนที่ตาม ROS และ Gazebo แสดงให้เห็นว่าหลังจากสร้างโมเดลหุ่นยนต์อย่างถูกต้องภายใต้ Gazebo แล้ว งานที่พัฒนาขึ้นสำหรับกระบวนการจำลองสามารถนำไปใช้กับหุ่นยนต์จริงได้โดยตรง โดยไม่ต้องดัดแปลง และนำเสนอการนำทางอัตโนมัติและการจำลองการทำแผนที่ 3 มิติ โดยใช้โปรแกรมควบคุมภายใต้ ROS ทั้งผลการจำลองและการทดลองสอดคล้องกันเป็นอย่างดีและแสดงให้เห็นถึงความสามารถในการใช้งานของสภาพแวดล้อมที่พัฒนาขึ้น

2.3.1 การใช้ ROS และ GAZEBO

ระบบปฏิบัติการหุ่นยนต์ (ROS) แพร่หลายและใช้โดยผู้ที่เกี่ยวข้องกับวิทยาการหุ่นยนต์ ระบบอนุญาตให้ใช้แพ็คเกจสำเร็จรูป เครื่องมือ แบบจำลอง และไลบรารีสำหรับหุ่นยนต์จำนวนมาก และเนื่องจากเป็นโอเพ่นซอร์ส ผู้ใช้แต่ละคนจึงมีความสามารถในการใช้และแก้ไขแพ็คเครื่องมือ หรือไลบรารีที่เหมาะสมสำหรับตน

เครื่องจำลอง Gazebo สามารถเชื่อมต่อกับ ROS ได้โดยตรงผ่านแพ็คเกจพิเศษ แพ็คเกจเหล่านี้มีอินเทอร์เฟซที่จำเป็นในการจำลองหุ่นยนต์ Gazebo โดยใช้ข้อความ ROS บริการ และการกำหนดค่าใหม่แบบไดนามิก เพื่อให้ทำงานกับหุ่นยนต์ได้อย่างถูกต้อง ROS จำเป็นต้องมีคำอธิบายเกี่ยวกับจลนศาสตร์ของหุ่นยนต์ ด้วยวิธีนี้ จึงสามารถวางแผนและดำเนินการเส้นทางโคจร การนำทาง และอื่น ๆ ได้ วิธีการอธิบายหุ่นยนต์ของ ROS คือการระบุคุณสมบัติในไฟล์ URDF (Universal Robot Description Format) URDF รองรับภาษา XML และ Xacro (XML macro) โค้ด Xacro ง่ายต่อการใช้งาน บำรุงรักษา และอ่านง่ายขึ้น หากต้องการใช้ไฟล์ URDF ใน Gazebo ต้องเพิ่มแท็กเฉพาะการจำลองเพิ่มเติมเพื่อให้ทำงานกับ Gazebo ได้อย่างถูกต้อง

ความสัมพันธ์ระหว่าง ROS และ Gazebo เหมือนกับความสัมพันธ์ระหว่าง ROS และฮาร์ดแวร์ของหุ่นยนต์จริง ตัวควบคุมใน ROS รับข้อมูลใน Topic และเผยแพร่ข้อมูลใน Topic ไปยังหุ่นยนต์เพื่อสั่งงาน ด้วยวิธีนี้ทำให้สามารถเปรียบเทียบพฤติกรรมและงานได้อย่างง่ายดาย

ข้อดี

ระบบปฏิบัติการหุ่นยนต์และแพลตฟอร์ม Gazebo มีความสามารถที่ดีมากสำหรับการสร้างและควบคุมหุ่นยนต์ การทดลองแสดงให้เห็นว่าโดยไม่ต้องทำการแก้ไขพิเศษกับการตั้งค่าการทำแผนที่ Localization และการนำทาง หุ่นยนต์สามารถเคลื่อนที่ได้อย่างอิสระในสภาพแวดล้อมจำลอง

ข้อด้อย

ระบบการติดตามเส้นทางยังไม่มีความแม่นยำ การเคลื่อนไหวที่ราบรื่น และตำแหน่งที่แม่นยำ ยังต้องเพิ่มระบบเพิ่มเติมเช่น ตัวควบคุมความเร็ว

บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง

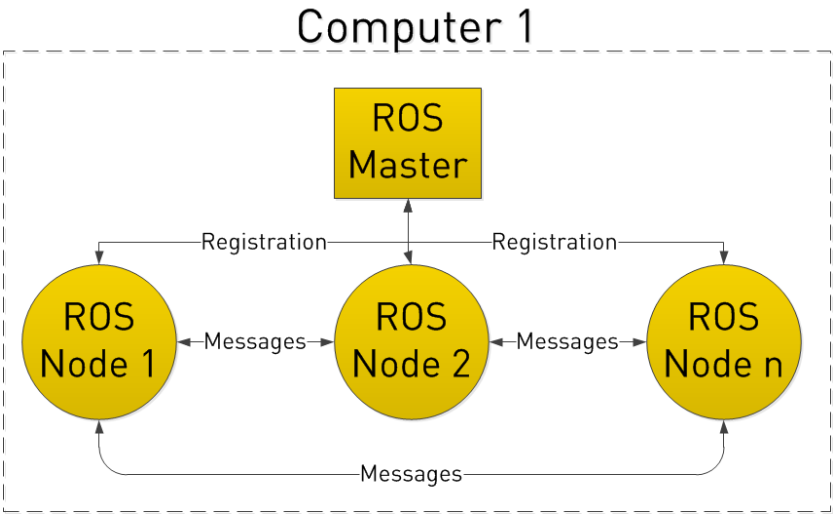
ในบทนี้จะอธิบายเกี่ยวกับความรู้ที่เกี่ยวข้อง ที่ถูกนำมาประยุกต์ใช้ในโครงงาน ผู้พัฒนาได้ทำการค้นคว้าทฤษฎีและความรู้ที่เกี่ยวข้องกับโครงงานที่นำมาใช้พัฒนาระบบได้ ดังนี้

3.1 Robot Operating System (ROS) [6]

Robot Operating System (ROS) เป็นโครงสร้างที่ยืดหยุ่นซึ่งสร้างขึ้นโดยมีจุดประสงค์เพื่อลดความซับซ้อนของการใช้งานและการเข้ารหัสซอฟต์แวร์หุ่นยนต์ เป็นการผสมผสานระหว่างเครื่องมือ ไลบรารี และข้อตกลงต่าง ๆ ที่มีวัตถุประสงค์หลักในการอำนวยความสะดวกในการสร้างพฤติกรรมของหุ่นยนต์ที่ซับซ้อนและแข็งแกร่งบนแพลตฟอร์มหุ่นยนต์ที่หลากหลาย

ROS เรียกกันทั่วไปว่าระบบปฏิบัติการ Meta เนื่องจากทำหน้าที่หลายอย่างของระบบปฏิบัติการ แต่ต้องใช้ระบบปฏิบัติการคอมพิวเตอร์ เช่น Linux มีจุดประสงค์เพื่อสื่อสารระหว่างระบบปฏิบัติการของคอมพิวเตอร์ อุปกรณ์ภายนอกคอมพิวเตอร์กับผู้ใช้ ประโยชน์หลักของ ROS คือการแยกฮาร์ดแวร์และความสามารถในการควบคุมหุ่นยนต์โดยที่ผู้ใช้ไม่จำเป็นต้องรู้รายละเอียดทั้งหมดของหุ่นยนต์

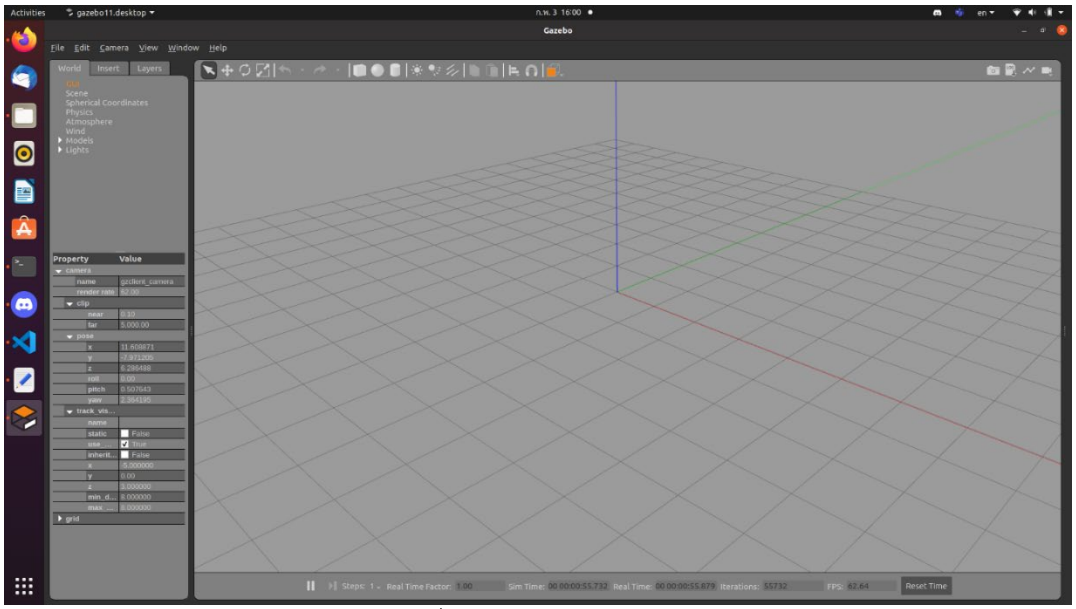
คำสั่ง ROS สามารถออกได้ โดยใช้รหัส Python ที่เขียนโดยโปรแกรมเมอร์ของหุ่นยนต์ ทำให้หุ่นยนต์สามารถดำเนินการตามที่สั่งได้ อย่างไรก็ตาม โค้ดเหล่านี้สามารถเรียกใช้โปรแกรมอื่น ๆ เพื่อดำเนินการได้ นี่คือข้อดีอย่างหนึ่งของ ROS การสื่อสารที่ไม่ซับซ้อนระหว่างแพลตฟอร์มและการลดความซับซ้อนของคำสั่งสำหรับผู้ใช้ ในเครือข่ายที่เชื่อมต่อกับเครื่องที่ใช้ระบบ ROS จะมีเครื่องหนึ่งที่จะทำหน้าที่เป็นต้นแบบเสมอ ต้นแบบต้องดำเนินการคำสั่ง roscore ซึ่งจะอนุญาตให้โหนดของ ROS ทั้งหมดสื่อสารกันได้ โดยเริ่มบริการที่จำเป็นสำหรับสิ่งนี้ โหนด ROS นั้นเป็นโปรแกรมปฏิบัติการ ดังนั้นจึงเป็นไปได้ว่าอาจมีโหนด ROS ทำงานบนเครื่องใดๆ บนเครือข่าย การสื่อสารระหว่างโหนดมักจะใช้ Transmission Control Protocol/Internet Protocol (TCP/IP) แต่ละ Node สามารถ Publish และ Subscribe ไปยัง Topics ต่าง ๆ Topics เหล่านี้ทำหน้าที่เป็นตัวแปร นั่นคือ Node สามารถเขียนค่าให้กับตัวแปรนี้ได้ แต่ก็สามารถอ่านค่าของมันได้เช่นกัน และนั่นคือเหตุผลว่าทำไมหลายๆ Nodeจึงสามารถสื่อสารกันในระบบ ROS ได้อย่างง่ายดายดังรูปที่ 3.1



รูปที่ 3.1 ตัวอย่างการทำงานของ ROS

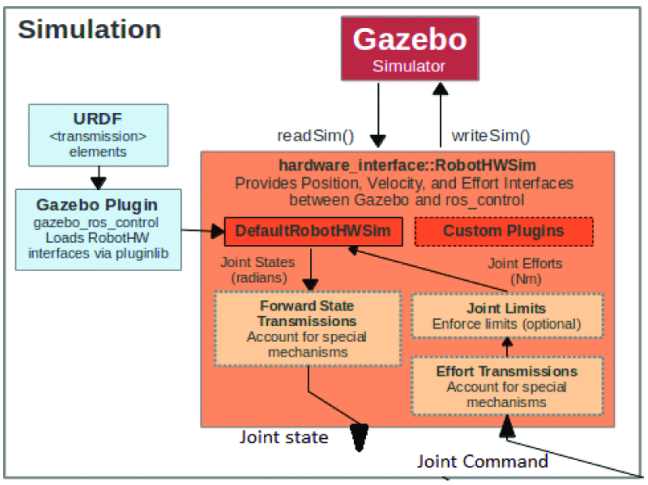
3.2 Gazebo [7]

Gazebo เป็นเครื่องจำลองหุ่นยนต์แบบโอเพ่นซอร์สที่ทำการจำลองแบบสามมิติ การจำลองหุ่นยนต์เป็นชิ้นส่วนพื้นฐานสำหรับทุกคนที่ต้องการทำงานด้านวิทยาการหุ่นยนต์ โปรแกรมจำลองที่ออกแบบมาอย่างดีช่วยให้ทดสอบอัลกอริทึม ออกแบบหุ่นยนต์ ทำการทดสอบการถดถอย และฝึกระบบปัญญาประดิษฐ์โดยใช้สถานการณ์จริงได้อย่างรวดเร็ว ซอฟต์แวร์ Gazebo ได้รับเลือกเนื่องจากมีความสามารถในการจำลองหุ่นยนต์ต่าง ๆ รวมถึง MiR 100 ด้วยความแม่นยำและประสิทธิภาพที่เหมาะสมในสภาพแวดล้อมในร่มและกลางแจ้งที่ซับซ้อน นอกจากนี้ยังเข้ากันได้กับระบบ ROS นอกจากนี้ยังมีกราฟิกคุณภาพสูงและอินเทอร์เฟซแบบกราฟิกและแบบโปรแกรมที่สะดวก ดังรูปที่ 3.2

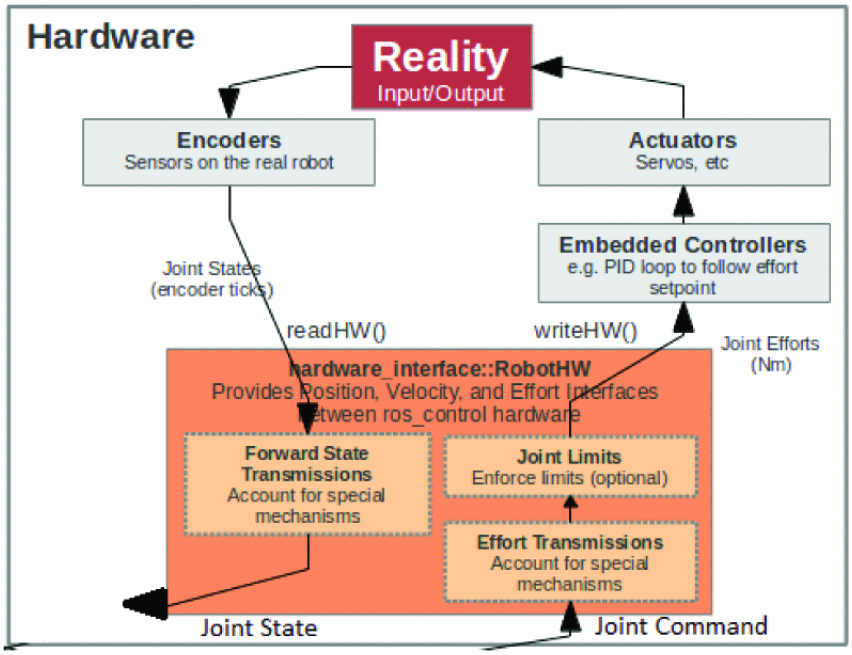


รูปที่ 3.2 โปรแกรม Gazebo

โดยการจำลองหุ่นยนต์ใน Gazebo นั้น จำเป็นต้องมีไฟล์ URDF โดย URDF นั้นคือไฟล์ที่กำหนดส่วนประกอบต่าง ๆ ของหุ่นยนต์ โดยกำหนดเป็นลิงค์ (link) และข้อต่อ (joint) ของแต่ละส่วนที่เชื่อมกัน และส่วนที่เรียกว่า Plugin ที่ทำหน้าที่ในการจำลองการทำงานแต่ละส่วน ตัวอย่างเช่น สร้าง link ที่เป็นทรงสี่เหลี่ยมขึ้นมา และกำหนดให้มี Plugin เป็นกล้อง ส่วนของ link นี้ก็จะสามารถทำหน้าที่จับภาพได้ ด้วย Plugin ของกล้อง และ Gazebo จะทำการอ่านค่าต่าง ๆ ที่ถูกกำหนดในไฟล์ URDF เพื่อทำการจำลองลงไปในโปรแกรม Gazebo ดังรูปที่ 3.3 และ 3.4



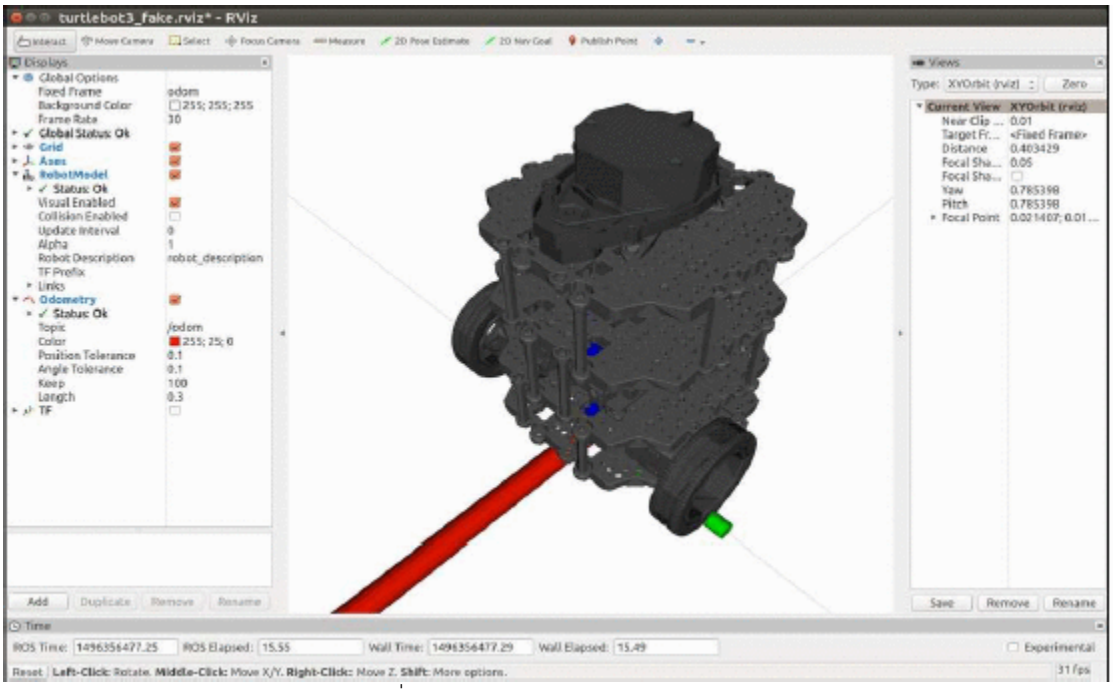
รูปที่ 3.3 การจำลองหุ่นยนต์หลักใน Gazebo



รูปที่ 3.4 ฮาร์ดแวร์หลักของหุ่นยนต์

3.3 Rviz [8]

Rviz (การแสดงภาพ ROS) เป็นโปรแกรมดูภาพสามมิติที่มีวัตถุประสงค์หลักเพื่อแสดงข้อมูล Sensor และข้อมูลสถานะ ROS เมื่อใช้ Rviz คุณสามารถดูการกำหนดค่าโมเดลหุ่นยนต์เสมือนปัจจุบันได้ นอกจากนี้ยังเป็นไปได้ที่จะแสดงค่า Sensor ที่แสดงสดในหัวข้อ ROS รวมถึงข้อมูลกล้อง การวัดระยะทางอินฟราเรด ข้อมูลโซนาร์ การวัด Sensor เลเซอร์ และการใช้งานอื่น ๆ อีกมากมายดังรูปที่ 3.5



รูปที่ 3.5 TurtleBot3 บนโปรแกรม Rviz

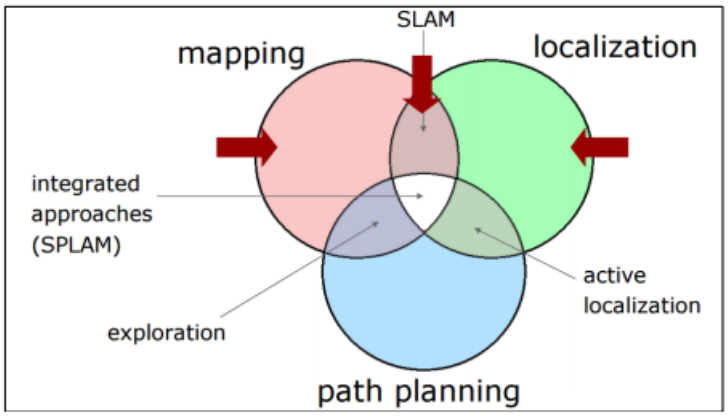
3.4 MiR 100 [9]

MiR Robot (Mobile industrial robots) เป็นหุ่นยนต์ที่ใช้แพร่หลายอุตสาหกรรม และ โรงพยาบาล โดยงานหลักที่ใช้คือ การจัดส่งยา การดูแลผู้ป่วย และฟังก์ชันการพยาบาลอื่น ๆ โดยหุ่นยนต์ถูกออกแบบให้มีหลายรุ่นตามงานที่เหมาะสม โดยROS มีโมเดล MiR 100 ให้ใช้ได้ทั้งระบบ ROS เวอร์ชัน Melodic และ Noetic ที่เป็นรุ่นล่าสุดของ ROS โดยหุ่นยนต์ Mir100 จะมีลักษณะรูปทรงดังรูปที่ 3.6

รูปที่ 3.6 โมเดลของ MiR 100

3.5 Autonomous Navigation [10]

การเคลื่อนที่และการนำร่อง หุ่นยนต์จำเป็นต้องเข้าใจสิ่งแวดล้อมและตำแหน่งและสภาวะของตนเอง ในสิ่งแวดล้อมนั้นๆ โดยอาศัยข้อมูลการวัดจาก Sensor ระบบนำร่องจึงจำเป็นต้องอาศัย 3 องค์ประกอบ ได้แก่ แผนที่ (Map) การระบุตำแหน่ง (Localization) การวางแผนเส้นทาง (Path Planning) ดังรูปที่ 3.7



รูปที่ 3.7 องค์ประกอบของ Autonomous Navigation

แผนที่ (Map) นำเสนอสิ่งแวดล้อมที่หุ่นยนต์กำลังทำงานอยู่ โดยบรรจุเอาองค์ประกอบของสิ่งแวดล้อมต่าง ๆ ที่หุ่นยนต์สนใจ หรือจำเป็นต่อการทำงานของหุ่นยนต์ ทั้งนี้แผนที่อาจอยู่ในรูปแบบที่หลากหลาย เช่น Metric Map ซึ่ง กำหนดกรอบอ้างอิง (Reference Frame) ซึ่งหุ่นยนต์ใช้ในการอ้างอิง เพื่อกำหนดตำแหน่งของตนจากกรอกอ้างอิง ดังกล่าว จากนั้นจึงสามารถคำนวณเส้นทาง (path) ซึ่งกำหนดแผนของการกระทำ (actions) เพื่อเคลื่อนที่ไปยัง เป้าหมาย

การระบุตำแหน่ง (Localization) คือการพิจารณาตำแหน่งปัจจุบันของหุ่นยนต์ โดยเทียบกับแผนที่

การวางแผนเส้นทาง (Path Planning) คือพิจารณาหาเส้นทาง และระบุชุดของการกระทำที่จะพาไปถึงจุดหมายปลายทาง โดยอาศัยแผนที่และการระบุตำแหน่ง

3.6 Simultaneous Localization and Mapping (SLAM) [11]

Simultaneous Localization and Mapping (SLAM) คือกระบวนการระบุตำแหน่งของหุ่นยนต์และสร้างแผนที่ทำ
โดยใช้ข้อมูลจากการวัดของ Sensors เพื่อใช้สภาพแวดล้อมรอบตัวในการประมาณแผนที่ของสิ่งแวดล้อม และวิถีการเคลื่อนที่ของหุ่นยนต์ SLAM เก็บข้อมูลการเคลื่อนที่ของตัวเองจาก Odometry และเก็บข้อมูล Landmark อันได้แก่ feature ต่าง ๆ ของสิ่งแวดล้อม เช่น มุมหรือสิ่งกีดขวาง จากการตรวจจับโดย Sensors โดยมีทั้งการเก็บ Landmark ใหม่และการสำรวจ Landmark เก่าที่เคยเก็บข้อมูลมาแล้ว ทั้งนี้ กระบวนการ SLAM ซึ่งต้องการกลไก เช่น Extend Kalman Filter (EKF) เพื่ออัปเดตข้อมูล Landmark และประมาณระดับความไม่แน่นอนของตำแหน่งหุ่นยนต์และข้อมูล Landmark ในสิ่งแวดล้อม โดยกระบวนการดังกล่าวจะกระทำไปเรื่อย ๆ ในขณะที่หุ่นยนต์เคลื่อนที่เพื่อให้ได้ข้อมูลเกี่ยวกับสิ่งแวดล้อม และสามารถนำมาสร้างเป็นแผนที่ได้ในที่สุด

3.7 Adaptive Monte Carlo Localization (AMCL) [12]

Adaptive Monte Carlo Localization (AMCL) หรือ Particle filter localization เป็นอัลกอริทึมสำหรับระบุตำแหน่งของหุ่นยนต์โดยใช้ particle filter จากแผนที่ที่มีอยู่ โดยอัลกอริทึมนี้จะใช้หลักการประมาณว่า สุ่มตัวหุ่นยนต์จำลองในแผนที่ขึ้นมาหลายๆ ตัวเรียกว่า particles หมายความว่าตอนนี้เรายังไม่รู้แน่ชัดว่าหุ่นยนต์ตัวจริงอยู่ตรงไหน อาจจะเป็นตัวใดตัวหนึ่งจากที่เราสุ่มขึ้นมาก็ได้ หลังจากนั้นจะสั่งให้หุ่นยนต์เคลื่อนที่ คราวนี้หุ่นยนต์ทุกตัวจะเคลื่อนที่ด้วยคำสั่งเดียวกัน โดยการเคลื่อนที่นี้จะไม่ได้สนใจว่า Sensor จะอ่านได้อย่างไร เหมือนกับปิดตาทำตามคำสั่ง เมื่อทำเสร็จแล้วคราวนี้หุ่นยนต์ทุกตัวก็จะอ่าน Sensor หุ่นยนต์ตัวไหนอ่าน Sensor แล้วใกล้เคียงกับ Sensor มากที่สุดก็มีความน่าจะเป็นว่าเป็นหุ่นยนต์ตัวจริงมากที่สุด ตัวที่เหลือก็ลบทิ้งไปแล้วก็สุ่มใหม่ให้ใกล้ๆ กับตัวหุ่นยนต์ที่มีความน่าจะเป็นสูงๆ และทำซ้ำไปเรื่อย ๆ

โดย AMCL มีความสำคัญอย่างมาก ในการทำให้หุ่นยนต์สามารถเคลื่อนที่ได้อย่างอัตโนมัติ เนื่องจากหากไม่สามารถระบุตำแหน่งที่แท้จริงของหุ่นยนต์ได้ จะทำให้หุ่นยนต์ไม่สามารถคิดวางแผนเส้นทางการเคลื่อนที่จากต้นทางไปยังปลายทางได้ เพราะ ไม่ทราบค่าตำแหน่งต้นทางที่ตัวเองกำลังอยู่ว่าตัวเองอยู่ตำแหน่งใด ในแผนที่

3.8 Sensor Robot [13]

Sensor Robot คือหุ่นยนต์เคลื่อนที่ติดตั้ง Sensor จำนวนมากที่ช่วยให้มองเห็นและรับรู้สภาพแวดล้อมได้ Sensor เหล่านี้ได้รับข้อมูลที่สามารถใช้ในการสร้างและบำรุงรักษาแผนที่ของสภาพแวดล้อม เพื่อจำกัดตำแหน่งหุ่นยนต์บนแผนที่ และเพื่อดูสิ่งกีดขวางในสภาพแวดล้อม งานเหล่านี้มีความสำคัญเพื่อให้สามารถนำทางหุ่นยนต์ผ่านสภาพแวดล้อมแบบไดนามิกได้อย่างปลอดภัยและมีประสิทธิภาพ

3.9 MySQL [14]

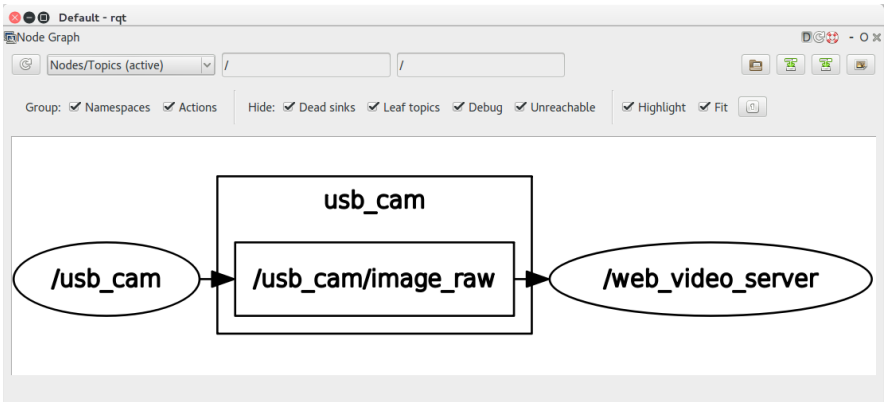
คือ ระบบจัดการฐานข้อมูล หรือ Database Management System (DBMS) แบบข้อมูลเชิงสัมพันธ์ หรือ Relational Database Management System (RDBMS) ซึ่งเป็นระบบฐานข้อมูลที่จัดเก็บรวบรวมข้อมูลในรูปแบบตาราง โดยมีการแบ่งข้อมูลออกเป็นแถว (Row) และในแต่ละแถวแบ่งออกเป็นคอลัมน์ (Column) เพื่อเชื่อมโยงระหว่างข้อมูลในตารางกับข้อมูลในคอลัมน์ที่กำหนด แทนการเก็บข้อมูลที่แยกออกจากกัน โดยไม่มีความเชื่อมโยงกัน ซึ่งประกอบด้วยข้อมูล (Attribute) ที่มีความสัมพันธ์เชื่อมโยงกัน (Relation)

3.10 Web Video Server [15]

Web Video Server เป็นแพ็กเกจซอฟต์แวร์ที่ใช้ในระบบ ROS (Robot Operating System) เพื่อสตรีมวิดีโอจากกล้องหรืออุปกรณ์รับภาพอื่น ๆ ไปยังเว็บเบราว์เซอร์ผ่านทางเครือข่าย ซึ่งช่วยให้ผู้ใช้สามารถดูวิดีโอสดหรือระบุการส่งคำสั่งไปยังระบบ ROS ผ่านเว็บเบราว์เซอร์ได้

Web Video Server ใช้เทคโนโลยีเว็บเบราว์เซอร์เพื่อสร้างเฟรมข้อมูลวิดีโอจากกล้องหรืออุปกรณ์รับภาพที่เชื่อมต่อกับระบบ ROS แล้วส่งข้อมูลเหล่านั้นผ่านทาง WebSocket ไปยังเว็บเบราว์เซอร์ ผู้ใช้งานสามารถเข้าถึงวิดีโอสดผ่านเว็บเบราว์เซอร์ของพวกเขา โดยรองรับการแสดงผลวิดีโอในรูปแบบที่เป็นที่นิยม เช่น MJPEG (Motion JPEG) หรือ H.264

Web Video Server เป็นเครื่องมือที่มีประโยชน์ในการพัฒนาและทดสอบระบบการควบคุมหุ่นยนต์หรือแอปพลิเคชันที่ใช้ภาพหรือวิดีโอใน ROS และช่วยให้ผู้ใช้งานสามารถเข้าถึงข้อมูลวิดีโอได้ง่ายๆ ผ่านทางเว็บเบราว์เซอร์ ดังรูปที่ 3.8



รูปที่ 3.8 การทำงานของ Web_Video_Server

3.11 Rosbridge_suite [16]

ROS Bridge_suite เป็นชุดเครื่องมือที่พัฒนาขึ้นสำหรับการเชื่อมต่อระหว่างระบบ ROS (Robot Operating System) กับระบบอื่น ๆ ที่ใช้เทคโนโลยีการสื่อสารที่แตกต่างออกไป เช่น ROS ไปยังเว็บเบราว์เซอร์หรือแอปพลิเคชันอื่น ๆ ที่ไม่ใช้ ROS โดยตรง

Bridge_suite ให้เครื่องมือสำหรับแปลงข้อมูลระหว่างระบบ ROS และระบบอื่น ๆ ซึ่งรวมถึงเครื่องมือสำหรับสร้างและส่งข้อมูลระหว่าง ROS และเว็บเบราว์เซอร์ หรือ ROS และแอปพลิเคชันอื่น ๆ ที่ใช้เทคโนโลยีอื่น ๆ เช่น JSON (JavaScript Object Notation) หรือ WebSocket ซึ่งช่วยให้ผู้พัฒนาสามารถใช้ ROS ร่วมกับระบบอื่น ๆ ที่เหมาะสมในงานที่ต้องการ ไม่ว่าจะเป็นการแสดงผลกราฟิกหรือการควบคุมโดยใช้แอปพลิเคชันที่ไม่ใช้ ROS

3.12 Roslibjs [17]

Roslibjs เป็นไลบรารี JavaScript ที่ใช้ในการเชื่อมต่อและสื่อสารกับ ROS (Robot Operating System) ผ่านทางเว็บเบราว์เซอร์ (Web Browser) หรือแอปพลิเคชันเว็บเบสด์ (Web-based Application) โดยใช้โปรโตคอล WebSocket ในการส่งข้อมูลระหว่างเบราว์เซอร์และเซิร์ฟเวอร์ ROS. สามารถสร้างและควบคุมโหนด (node) ของ ROS ในเว็บเบราว์เซอร์ได้ รวมถึงส่งข้อมูลไปยังโหนดอื่น ๆ ในระบบ ROS และรับข้อมูลจากโหนดอื่น ๆ ด้วย WebSocket ซึ่งเป็นทางการสื่อสารระหว่างเบราว์เซอร์และเซิร์ฟเวอร์ROS Roslibjs มีฟังก์ชันที่มีประโยชน์มากมาย เช่น การสร้างตัวแปรและออบเจ็กต์ใน ROS เพื่อส่งข้อมูลและรับข้อมูลระหว่างเบราว์เซอร์และเซิร์ฟเวอร์ ROS ตัวอย่างเช่น การกำหนดตัวแปร robotListener ให้เป็น

ออบเจ็คต์ในการชี้ไปที่ Topic ที่ชื่อว่า /mir/base_pose_ground_truth และมี Typeของข้อความเป็น nav_msgs/Odometry เพื่อใช้ในการติดตามการเคลื่อนที่ของหุ่นยนต์ ดังรูปที่ 3..9

```
var robotListener = new ROSLIB.Topic({
  ros: ros,
  name: '/mir/base_pose_ground_truth',
  messageType: 'nav_msgs/Odometry'
});
robotListener.subscribe(function (message) {
  var position = new ROSLIB.Vector3();
  position.x = message.pose.pose.position.x;
  position.y = message.pose.pose.position.y;
  position.z = message.pose.pose.position.z;
  robotMarker.x = position.x;
  robotMarker.y = -position.y + 1.7;
  robotMarker.rotation = message.pose.pose.orientation.z;
});
```

รูปที่ 3.9 ตัวอย่างโค้ด Roslibjs

3.13 Ros2djs [18]

ros2djs ให้เครื่องมือและอินเตอร์เฟซที่ทำให้เราสามารถสร้างและควบคุมโหนด ROS ในเว็บเบราว์เซอร์ได้ รวมถึงส่งข้อมูลและรับข้อมูลระหว่างเบราว์เซอร์และโหนด ROS โดยอาศัยการสื่อสารแบบเรียลไทม์ระหว่างเซิร์ฟเวอร์ ROSและเว็บเบราว์เซอร์ผ่านสิ่งที่เรียกว่า ROS Bridge โดย ros2djs ต้องพึ่งพาการทำงานร่วมกับ Roslibjs EventEmitter2 และ EaseUS

ไลบรารีนี้มีความสามารถที่หลากหลาย เช่น การสร้างแผนที่ การแสดงผลเส้นทาง (path) การแสดงข้อมูล Sensor การควบคุมหุ่นยนต์ การสร้างสัญลักษณ์ (marker) และอื่น ๆ เพื่อช่วยในการพัฒนาแอปพลิเคชันที่ใช้ร่วมกับ ROS ตัวอย่างเช่น การนำภาพจาก Topic /map ขึ้นมาแสดงบนเว็ป และการกำหนด สัญลักษณ์ (marker) ของหุ่นยนต์ ลงในแผนที่ เพื่อกำหนดการติดตามแบบเรียลไทม์ ดังรูปที่ 3.10 และ 3.11

```
var viewer = new ROS2D.Viewer({
  divID: 'map',
  width: 640,
  height: 640,
});
var gridClient = new ROS2D.OccupancyGridClient({
  ros: ros,
  rootObject: viewer.scene,
  continuous: true
});
gridClient.on('change', function () {
  viewer.scaleToDimensions(gridClient.currentGrid.width, gridClient.currentGrid.height);
  viewer.shift(gridClient.currentGrid.pose.position.x, gridClient.currentGrid.pose.position.y);
});
```

รูปที่ 3.10 ตัวอย่างโค้ด การนำภาพจาก /map ขึ้นเว็บด้วย ros2djs

```
var robotMarker = new ROS2D.NavigationArrow({
  size: 0.001,
  strokeSize: 0.01,
  pulse: true,
});
robotMarker.graphics.beginFill("#FF0000");
robotMarker.graphics.moveTo(0, 0);
robotMarker.graphics.lineTo(1, 0.5);
robotMarker.graphics.lineTo(0, 1);
robotMarker.graphics.closePath();
robotMarker.graphics.endFill();
```

รูปที่ 3.11 ตัวอย่างโค้ด การทำสัญลักษณ์ติดตามหุ่นยนต์บนแผนที่เว็บ

3.14 EvenEmitter2 [19]

EventEmitter2 เป็นไลบรารี JavaScript ที่ใช้ในการจัดการเหตุการณ์และการส่งออกข้อความระหว่างคอมโพเนนต์ในรูปแบบของอีเวนต์ สามารถสร้างอีเวนต์ และฟังก์ชันตอบสนอง (listener) ที่เชื่อมโยงกัน ฟังก์ชันตอบสนองจะถูกเรียกเมื่อเกิดเหตุการณ์ และสามารถส่งข้อมูลเพิ่มเติมผ่านพารามิเตอร์ของเหตุการณ์ได้ ซึ่งช่วยให้คุณสร้างระบบสื่อสารระหว่างคอมโพเนนต์หรือส่วนของโปรแกรมที่แตกต่างกันได้อย่างมีประสิทธิภาพและยืดหยุ่น

3.15 EaseUs [20]

EaseUS เป็นไลบรารี JavaScript ที่เกี่ยวข้องกับกราฟิกและการแสดงผลในเว็บเบราว์เซอร์ เป็นส่วนหนึ่งของ CreateJS ซึ่งเป็นชุดเครื่องมือสำหรับการพัฒนาเกมและแอนิเมชันในเว็บเบราว์เซอร์ EaseUS สามารถสร้างและควบคุมกราฟิกในเว็บเบราว์เซอร์ วาดรูปร่างพื้นฐาน เช่น รูปสี่เหลี่ยม วงกลม เส้น และอื่น ๆ และควบคุมการแสดงผล การเคลื่อนไหว การเปลี่ยนรูปแบบ การเชื่อมต่อกับเหตุการณ์ และการจัดรูปแบบองค์ประกอบกราฟิกต่าง ๆ ในหน้าเว็บ

บทที่ 4 ขั้นตอนและแผนการดำเนินงาน

บทนี้จะอธิบายถึงรายละเอียด ขั้นตอนการดำเนินงาน โดยกล่าวถึงรายละเอียดขั้นตอนการดำเนินงาน และ ตารางแผนการดำเนินงานและเวลาที่ใช้

4.1 ขั้นตอนการดำเนินงาน

4.1.1 การรวบรวมข้อมูลและกำหนดขอบเขต

- เลือกหัวข้อที่สนใจและปรึกษาอาจารย์ที่ปรึกษา
- รวบรวมข้อมูลที่เกี่ยวข้องกับโครงงาน
- กำหนดขอบเขตในการทำโครงงาน
- วางแผนการดำเนินงาน

4.1.2 การศึกษาและเรียนรู้ทฤษฎีที่เกี่ยวข้อง

- ศึกษาทฤษฎีที่เกี่ยวข้อง
- ศึกษาการเขียน Python Scripts
- ศึกษาการเขียน JavaScript
- ศึกษาการเขียน HTML CSS
- ศึกษาการใช้งาน Gazebo
- ศึกษาการใช้งาน ROS
- ศึกษาเกี่ยวกับการจัดการฐานข้อมูล

4.1.3 การวิเคราะห์และออกแบบระบบ

- วิเคราะห์การทำงานของระบบ
- ออกแบบระบบของการทำงาน

4.1.4 การพัฒนาระบบ

- สร้าง User Interface ด้วยภาษา HTML CSS JavaScript
- เขียนคำสั่งหุ่นยนต์ด้วยภาษา Python
- เขียนคำสั่งแสดงผลและโต้ตอบกับผู้ใช้งาน

4.1.5 การทดสอบระบบและปรับปรุง

- ตรวจสอบความถูกต้องของระบบ เพื่อนำไปแก้ไขและพัฒนาต่อไป

4.1.6 การจัดทำเอกสารประกอบโครงงาน

- จัดทำเอกสารประกอบโครงงาน อธิบายการทำงานและภาพรวมของการจำลองหุ่นยนต์

4.2 แผนการดำเนินงาน

ตารางที่ 4.1 แผนการดำเนินงาน

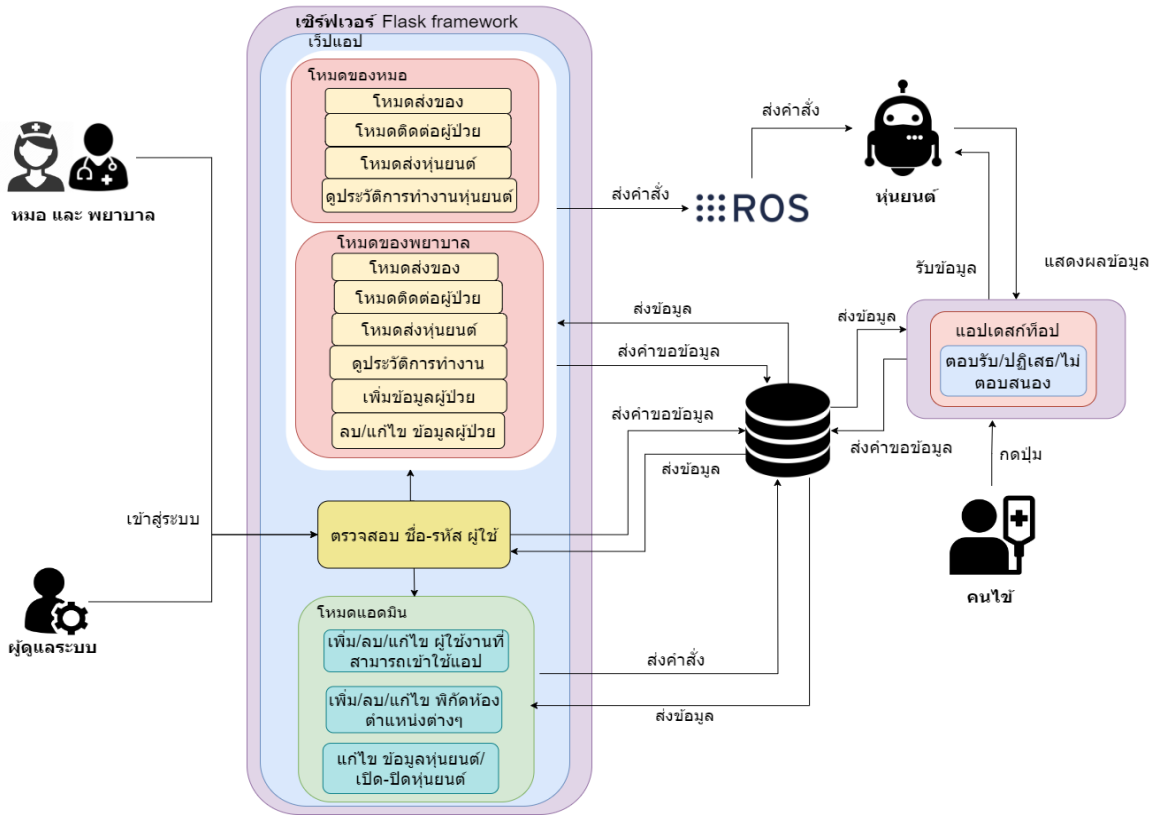
ขั้นตอนการดำเนินงาน	เดือน						
	ธ.ค. 2565	ม.ค. 2566	ก.พ. 2566	มี.ค. 2566	เม.ย. 2566	พ.ค. 2566	มิ.ย. 2566
การรวบรวมข้อมูลและกำหนดขอบเขต	↔						
การศึกษาและเรียนรู้ทฤษฎีที่เกี่ยวข้อง	↔	↔	↔				
การวิเคราะห์และออกแบบระบบ		↔	↔	↔			
การพัฒนาระบบ		↔	↔	↔	↔	↔	↔
การทดสอบระบบและประปรุง				↔	↔	↔	↔
การจัดทำเอกสารประกอบโครงงาน	↔	↔	↔	↔	↔	↔	↔

บทที่ 5 การวิเคราะห์และออกแบบระบบ

โดยบทนี้นี้จะอธิบายถึงกระบวนการทำงานของระบบ โครงสร้างของระบบ โครงสร้างของหุ่นยนต์ อธิบายระบบฐานข้อมูล และผู้ใช้งานประเภทต่าง ๆ ดังนี้

5.1 โครงสร้างของระบบ

โดยเว็ปแอปพลิเคชัน จะประกอบด้วยผู้ใช้ 3 กลุ่ม คือ หมอ พยาบาล และ ผู้ดูแลระบบ ในส่วนของหมอและพยาบาลมีหมวดการทำงานที่คล้ายคลึงกันคือการใช้งานหุ่นยนต์ โดยมีโหมดสั่งหุ่นยนต์ทั้งหมด 3 โหมด คือ ส่งของ ติดต่อผู้ป่วย และ ส่งหุ่นยนต์ แต่แตกต่างกันที่ พยาบาลจะมีเพิ่มจากหมอ 2 โหมด ในการเพิ่มข้อมูลผู้ป่วยที่เข้าพัก และลบแก้ไขข้อมูลผู้ป่วย เมื่อผู้ใช้งานสั่งงานหุ่นยนต์ คำสั่งจะถูกส่งไปยัง ROS เพื่อให้ ROS ทำหน้าที่ในการประสานงานกับหุ่นยนต์ โดยเว็ปแอปพลิเคชันจะใช้ Flask เข้ามาช่วยในการทำงานร่วมกันระหว่าง เว็ปแอปพลิเคชัน กับ ROS ที่ใช้ภาษา Python ในการควบคุมการทำงานของหุ่นยนต์ ในส่วน Database มี MySQL เป็นฐานข้อมูลในการเก็บข้อมูล โดยมีการเก็บประวัติการทำงานของหุ่นยนต์ ประวัติผู้ป่วยที่เข้าพักรักษา และ ตำแหน่งพิกัดต่าง ๆ ที่หุ่นยนต์ใช้ในการเคลื่อนที่ ในส่วนของหุ่นยนต์ หุ่นยนต์จะมีการเคลื่อนที่ ที่เป็นการเคลื่อนที่ในรูปแบบอัตโนมัติโดยใช้ MoveBase ในส่วนของผู้ดูแลระบบ สามารถเข้าใช้งานเว็ปแอปพลิเคชันได้ในส่วนของผู้ดูแลระบบได้ โดยมีหมวดการทำงานคือสามารถเพิ่มลบแก้ไขผู้มีสิทธิเข้าใช้งานเว็ปแอปพลิเคชันได้ เพิ่มลบแก้ไข พิกัดที่หุ่นยนต์ใช้ในการเคลื่อนที่ และ แก้ไขข้อมูลของหุ่นยนต์ และในส่วนสุดท้ายคือหน้าจอของหุ่นยนต์ ที่แสดงหน้าต่างแสดงการตอบรับการติดต่อระหว่างผู้ป่วยกับหุ่นยนต์ โดยเป็นเดสก์ท็อปแอปพลิเคชันที่จะแสดงขึ้นเมื่อหุ่นยนต์ถึงจุดหมายในการเข้าติดต่อกับผู้ป่วย ดังรูปที่ 5.1



รูปที่ 5.1 โครงสร้างการทำงานของระบบ

5.2 หลักการทำงานของระบบ

ระบบจะประกอบไปด้วย 3 ส่วน ได้แก่

5.2.1 การทำงานของหุ่นยนต์

หุ่นยนต์จะมีการทำงานที่แตกต่างกันทั้งหมด 3แบบ คือ

1) **ส่งของ** การทำงานในหมวดนี้ หุ่นยนต์จะรอรับคำสั่งที่เข้ามาสามารถสั่งได้พร้อมกัน 2 คิวงาน/ครั้ง โดยหุ่นยนต์จะนำของที่ได้รับมาไปส่งยังจุดที่ได้รับมอบหมาย เมื่อหุ่นยนต์ถึงจุดหมายจะมีหน้าจอบนหุ่นยนต์แสดงหน้าจอให้ผู้รับสามารถกดตอบรับได้จะมีทั้งหมด 3 กรณีดังนี้

กรณีที่ 1 ผู้ใช้งานกดปุ่มตกลงที่จะรับของที่มาส่ง กระจกของหุ่นยนต์ที่ปิดชั้นวางของอยู่ จะเปิดออกโดยอัตโนมัติเพื่อให้ผู้ป่วยสามารถหยิบของออกจากชั้นวางของได้และจะมีการบันทึกสถานะการทำงานว่าสำเร็จ เมื่อเสร็จสิ้นการทำงานหุ่นยนต์จะเคลื่อนที่กลับไปยังจุดเริ่มต้น

กรณีที่ 2 ผู้ใช้งานกดปุ่มปฏิเสธที่จะรับของที่มาส่ง จะมีการบันทึกสถานะการทำงานว่าไม่สำเร็จและหุ่นยนต์จะเคลื่อนที่กลับไปยังจุดเริ่มต้น

กรณีที่ 3 ไม่มีการตอบสนอง ในกรณีที่หุ่นยนต์มาส่งของ แต่ไม่มีการตอบรับใดๆ จากผู้ป่วย หุ่นยนต์จะรอเป็นเวลา 1 นาที และถ้าหากหมดเวลา จะมีการบันทึกสถานะว่า ไม่มีการตอบสนอง และหุ่นยนต์จะทำการเคลื่อนที่กลับไปยังจุดเริ่มต้น

2) **ติดต่อผู้ป่วย** การทำงานในหมวดนี้หุ่นยนต์จะรอรับคำสั่งที่เข้าสามารถมาสั่งได้ 1 คิวงาน/ครั้ง โดยหุ่นยนต์ จะไปยังจุดที่ได้รับมอบหมาย เมื่อหุ่นยนต์ถึงจุดหมายก็จะมีหน้าจอแสดงให้ป่วยกดตอบรับได้ โดยจะมีทั้งหมด 3 กรณีดังนี้

กรณีที่ 1 กดปุ่มตกลง หุ่นยนต์จะทำการหยุดรอให้ผู้ป่วยได้คุยกับหมอ พยาบาล โดยในส่วนของผู้ป่วยจะสามารถเห็นหมอและพยาบาลได้ ผ่านทางหน้าจอของหุ่นยนต์ และในส่วนของหมอและพยาบาลจะสามารถเห็นผู้ป่วยได้ผ่านเว็ปแอปพลิเคชันโหมดติดต่อผู้ป่วย ในส่วนของหมอและพยาบาลจะมีปุ่มวางสายให้สามารถกดได้เพื่อเสร็จสิ้นการทำงาน เมื่อผู้ใช้กดปุ่มวางสายการติดต่อ หุ่นยนต์จะทำการบันทึกสถานะการทำงานว่าสำเร็จและจะเคลื่อนที่กลับมายังจุดเริ่มต้น

กรณีที่ 2 กดปุ่มปฏิเสธ หุ่นยนต์จะบันทึกสถานะการทำงานว่าไม่สำเร็จ และจะเคลื่อนที่กลับมายังจุดเริ่มต้น

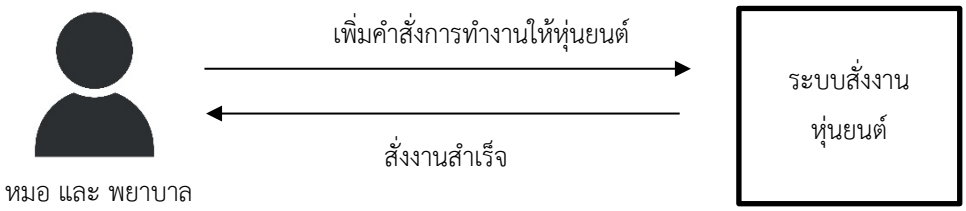
กรณีที่ 3 ไม่มีการตอบสนอง ในกรณีที่หุ่นยนต์มาติดต่อผู้ป่วย แต่ไม่มีการตอบรับใดๆ จากผู้ป่วย หุ่นยนต์จะรอเป็นเวลา 1 นาที และถ้าหากหมดเวลา จะมีการบันทึกสถานะว่า ไม่มีการตอบสนอง และหุ่นยนต์จะทำการเคลื่อนที่กลับไปยังจุดเริ่มต้น

3) **ส่งหุ่นยนต์** การทำงานในหมวดนี้ หุ่นยนต์จะรอรับคำสั่งที่เข้ามา สามารถสั่งได้ 1 คิวงาน/ครั้ง โดยหุ่นยนต์ จะทำการเคลื่อนที่ไปยังจุดหมายที่ได้รับเพียงอย่างเดียว ไม่มีการกลับายังจุดเริ่มต้น และเมื่อหุ่นยนต์ถึงจุดหมายจะถือว่าเสร็จสิ้นการทำงาน และจะกลับไปรอรับคำสั่งอีกครั้ง

5.2.2 ระบบติดต่อกับผู้ใช้งาน

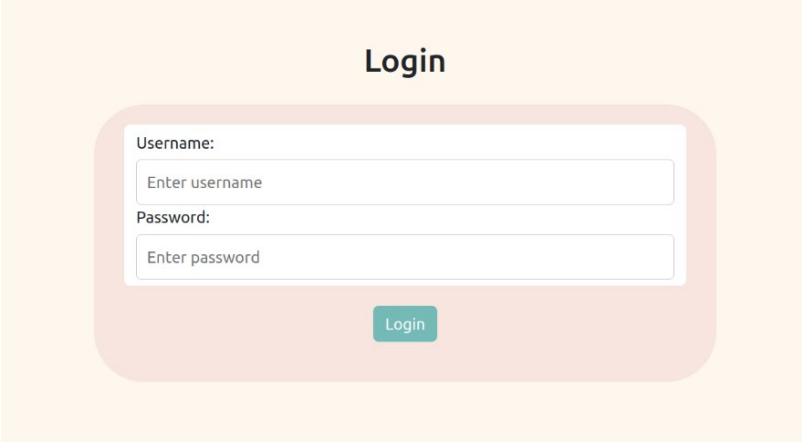
มีการทำงานอยู่ 2 ส่วน คือ

1. การสั่งงานของหมอและพยาบาล โดยมีการแบ่งโหมดการทำงาน 3 แบบ คือ ส่งของ ติดต่อผู้ป่วย และ ส่งหุ่นยนต์ไปยังที่ต่าง ๆ ดังรูป 5.2



รูปที่ 5.2 การทำงานระหว่างหมอและพยาบาล กับหุ่นยนต์

โดยหมวดการทำงานของหมอและพยาบาล ต้องทำการเข้าสู่ระบบก่อนดังรูปที่ 5.3 จึงสามารถเข้าสู่ตัวแอปพลิเคชันได้ โดยการทำงานมีทั้งหมด 3 หมวดดังนี้



รูปที่ 5.3 หน้า UI เข้าสู่ระบบการสั่งการหุ่นยนต์

1.ส่งของไปยังผู้ป่วย สำหรับต้องการส่ง ยา อาหาร หรืออื่น ๆ ไปยังผู้ป่วย โดยเมื่อหุ่นยนต์ทำงานจนเสร็จ จะกลับมายังจุดเริ่มต้น ดังรูปที่ 5.4

รูปที่ 5.4 หน้า UI ของหมอและพยาบาลในโหมดส่งของ

การเพิ่มคำสั่งงานให้หุ่นยนต์สามารถสั่งได้2งาน มีขั้นตอนดังนี้

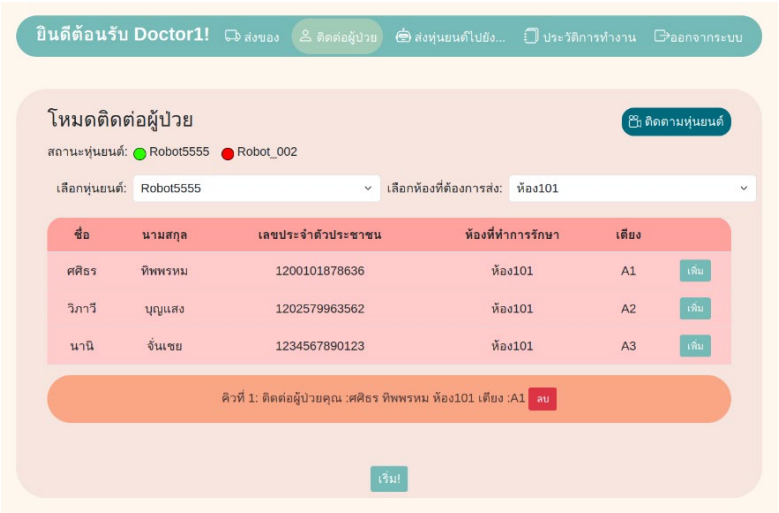
1. เลือกหุ่นยนต์ที่ต้องการสั่ง
2. เลือกชั้นวางของที่ต้องการใส่ของ
3. เลือกสิ่งของที่ต้องการส่ง
4. กรอกรายละเอียด เกี่ยวกับของที่ส่งหรือรายละเอียดของงาน
5. เลือกเตียงที่ต้องการส่งของ เพิ่มไปยังคิวงานของหุ่นยนต์
6. ถ้าต้องการสั่งหุ่นยนต์เพิ่มทำข้อ2-4อีกครั้ง
7. กดเริ่มการทำงานหุ่นยนต์

การแก้ไข คิวการทำงาน

1. กดปุ่ม ลบ รายการงานที่ต้องการยกเลิก ดังรูปที่ 5.5

รูปที่ 5.5 หน้า UI โหมดส่งขอซึ่งแสดงปุ่มลบ

2.ติดต่อผู้ป่วย สำหรับสั่งหุ่นยนต์ไปหาผู้ป่วย เพื่อต้องการติดต่อ ดูอาการของผู้ป่วย โดยจะมีกล้องจากหุ่นยนต์ เพื่อให้หมอ และพยาบาล ได้เห็นคนไข้ดังรูปที่ 5.6



รูปที่ 5.6 หน้า UI โหมดติดต่อผู้ป่วย

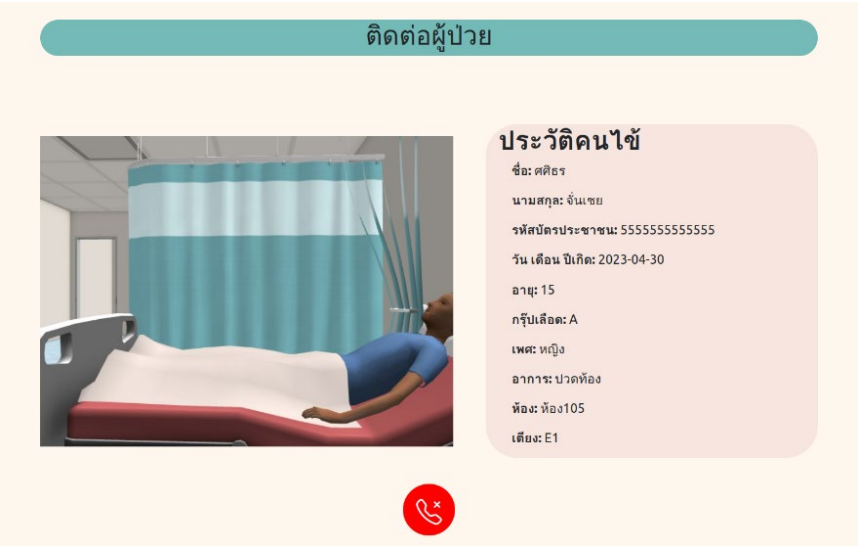
การเพิ่มคำสั่งงานให้หุ่นยนต์สามารถสั่งได้1งาน มีขั้นตอนดังนี้

1. เลือกหุ่นยนต์ที่ต้องการสั่ง
2. เลือกเตียงที่ต้องการติดต่อ
3. กดเริ่มการทำงานหุ่นยนต์

การแก้ไข คิวการทำงาน

1. กดปุ่ม ลบ รายการงานที่ต้องการยกเลิก ดังรูป 5.5

โดยหลังจากเริ่มสั่งหุ่นยนต์ หน้าจอ UI จะถูกเปลี่ยนเป็นข้อมูลประวัติคนไข้ และภาพจากกล้องหุ่นยนต์ที่เราสั่ง โดยหุ่นยนต์จะกลับมายังจุดเริ่มต้น หรือเสร็จสิ้นการติดต่อได้ ก็ต่อเมื่อผู้ใช้กดปุ่มวางสายสีแดงด่านล่าง ดังรูปที่ 5.7



รูปที่ 5.7 หน้า UI โหมดติดต่อผู้ป่วยหลังกด สั่งทำงานหุ่นยนต์

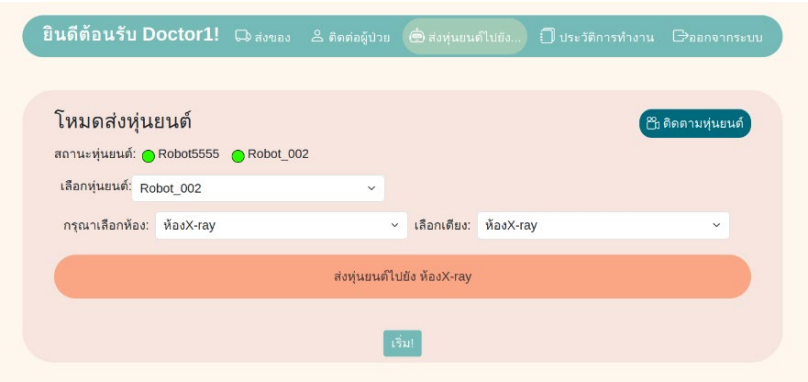
3.ส่งหุ่นยนต์ สำหรับต้องการส่งหุ่นยนต์ไปยังที่ต่าง ๆ โดยการทำงาน หุ่นยนต์จะเคลื่อนที่ไปยังจุดที่ได้รับมอบหมายเพียงอย่างเดียว จะไม่กลับมายังจุดเริ่มต้น

การเพิ่มคำสั่งงานให้หุ่นยนต์สามารถสั่งได้1งาน โดยสมารถสั่งผ่านหน้า UI ดังรูปที่ 5.8 มีขั้นตอนดังนี้

1. เลือกหุ่นยนต์ที่ต้องการสั่ง
2. เลือกตำแหน่งที่ต้องการส่งหุ่นยนต์ไป
3.กดเริ่มการทำงานหุ่นยนต์

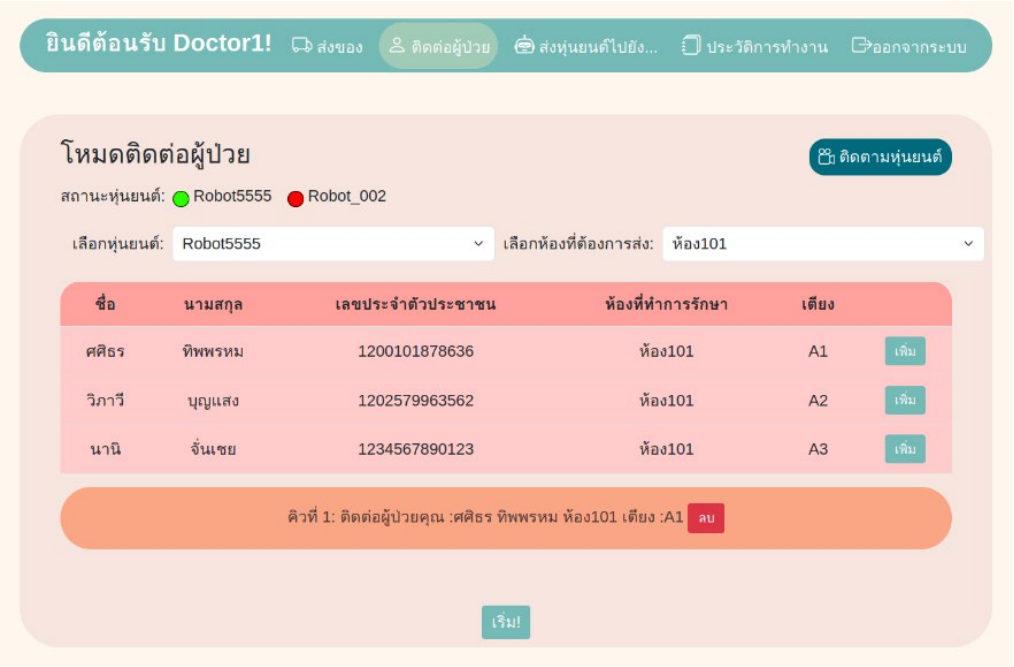
การแก้ไข คิวการทำงาน

1. กดปุ่ม ลบ รายการงานที่ต้องการยกเลิก ดังรูป 5.5



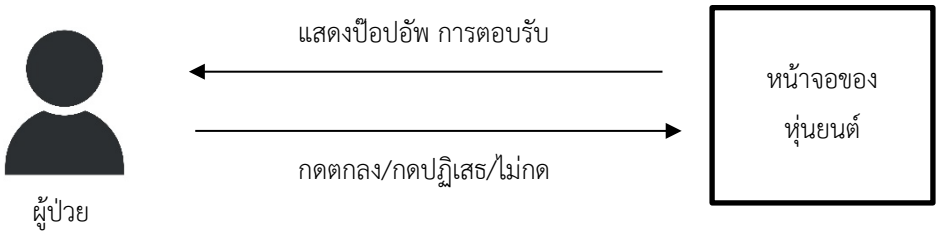
รูปที่ 5.8 หน้า UI โหมดส่งหุ่นยนต์

โดยทั้ง 3 โหมดนั้น จะสั่งงานหุ่นยนต์ได้ก็ต่อเมื่อหุ่นยนต์ที่สั่ง อยู่ในสถานะพร้อมใช้งานเท่านั้น โดยสามารถ ดูได้จากหน้า UI ของทุกโหมด เป็นสถานะของหุ่นยนต์โดยมีสีสถานะอยู่ 2 สีคือ สีเขียวเมื่อหุ่นยนต์พร้อมใช้งาน และสีแดงเมื่อหุ่นยนต์ไม่พร้อมใช้งาน ดังรูปที่ 5.9



รูปที่ 5.9 ภาพตัวอย่างสถานะหุ่นยนต์ พร้อมใช้งานและไม่พร้อมใช้งาน

2. การตอบรับของผู้ป่วยเมื่อหุ่นยนต์มายังเตียงของผู้ป่วย โดยสามารถตอบตกลง ปฏิเสธ หรือ ปล่อยให้หุ่นยนต์กลับไปเอง โดยภาพแสดงความสัมพันธ์ระหว่างผู้ป่วยกับหุ่นยนต์จะเป็นดังรูปที่ 5.10



รูปที่ 5.10 การตอบสนองระหว่างผู้ป่วยกับหุ่นยนต์

การตอบรับระหว่างผู้ป่วยกับหุ่นยนต์ที่มาส่งของหรือติดต่อผู้ป่วย โดยเมื่อหุ่นยนต์มาถึงจะมีหน้าจอบนหุ่นยนต์แสดงให้ผู้ป่วยสามารถทำการกดปุ่มตกลงเพื่อตอบรับของที่มาส่งหรือติดต่อ หรือสามารถกดปฏิเสธถ้ามีการส่งแล้วข้อมูลไม่ตรงกับตัวผู้รับ หรือ ไม่กดทั้ง 2 หุ่นยนต์ก็จะรอเป็นเวลา1นาที ถ้าไม่มีการตอบรับใดๆ จะถือว่าผู้รับปฏิเสธการส่ง

5.2.3 ระบบฐานข้อมูล

Back-end มีการทำงานหลักคือ Database โดยใช้ MySQL เป็นฐานข้อมูลในการเก็บข้อมูลต่าง ๆ ที่จำเป็นต้องใช้และเก็บบันทึก โดยมีทั้งหมด 8 ตาราง ดังรูปที่ 5.11

1) ตาราง patient ทำการเก็บประวัติคนไข้ที่เข้าพักในโรงพยาบาล

2) ตาราง bed ทำหน้าที่ในการเก็บชื่อเตียงและพิกัดตำแหน่งของเตียง ที่หุ่นยนต์ใช้ในการวางแผนไปยังตำแหน่งนั้นๆ

3) ตาราง Room ทำหน้าที่ในการเก็บชื่อห้อง และ List รายการเตียงที่อยู่ภายในห้อง

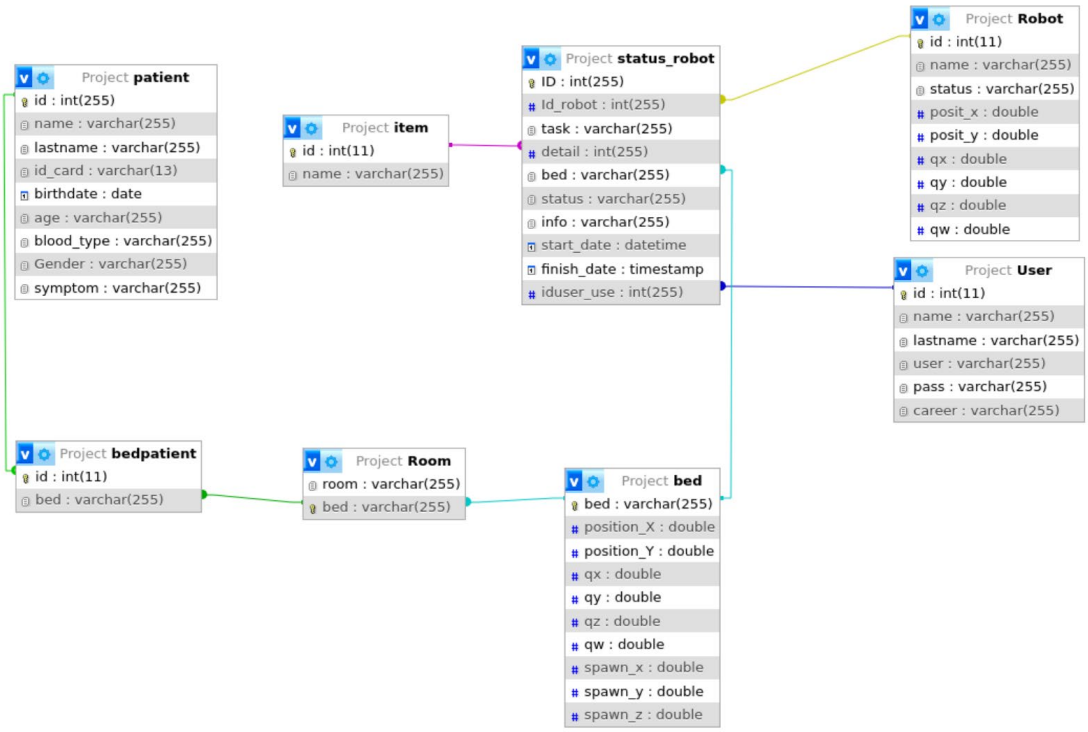
4) ตาราง bedpatient ทำหน้าที่ในการเก็บข้อมูลผู้ป่วยและห้องที่พัก ว่าผู้ป่วยเข้าพักอยู่ที่ห้องไหน

5) ตาราง Robot ทำหน้าที่ในการเก็บรายละเอียดของหุ่นยนต์ ชื่อ พิกัดจุดเริ่มต้นที่หุ่นยนต์เริ่มการทำงานและใช้เคลื่อนที่กลับมายังจุดเดิม

6) ตาราง User ทำหน้าที่ในการเก็บ ชื่อผู้ใช้ รหัสผ่าน สำหรับ User ที่มีสิทธิเข้าใช้งานเว็ปแอปพลิเคชัน

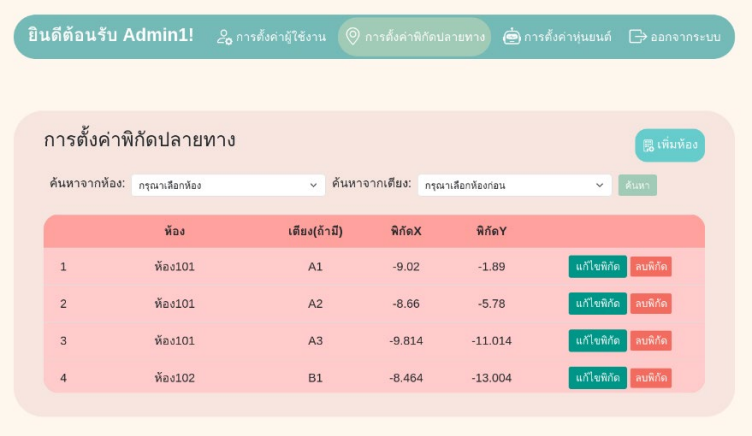
7) ตาราง item ทำหน้าที่ในการเก็บรายการสิ่งของที่สามารถส่งไปยังผู้ป่วยได้

8) ตาราง status_robot ทำหน้าที่ในการเก็บประวัติการทำงานทั้งหมดของหุ่นยนต์ที่ถูกสั่งงาน สถานะการทำงานของหุ่นยนต์ว่าทำงานในโหมดอะไร สำเร็จหรือไม่ ไปยังตำแหน่งไหน หรือใครเป็นผู้สั่งงานเป็นต้น



รูปที่ 5.11 ตารางข้อมูลที่ใช้ในการทำงานด้วย MySQL

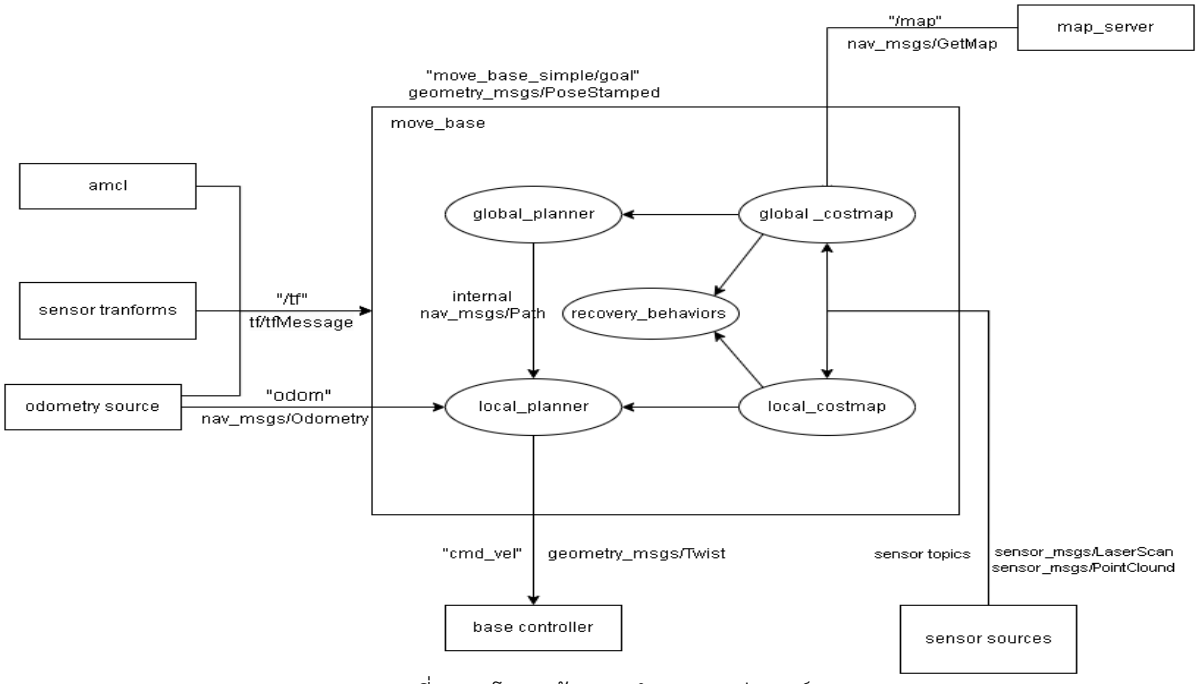
ผู้พัฒนาระบบ ทำหน้าที่ในการตั้งค่าเพิ่มลบแก้ผู้ที่จะมาใช้งานของระบบ ตั้งค่าพิกัดปลายทางของหุ่นยนต์ และทำหน้าที่ตั้งค่าการทำงานของหุ่นยนต์ทั้งหมด การเปลี่ยนชื่อหุ่นยนต์ เปิด/ปิดหุ่นยนต์ และแก้ไขพิกัดเริ่มต้นของหุ่นยนต์ ดังรูปที่ 5.12



รูปที่ 5.12 หน้า UI ของผู้พัฒนาระบบ

5.3 โครงสร้างการทำงานของหุ่นยนต์

โดยหุ่นยนต์จะเคลื่อนที่โดยอัตโนมัติได้นั้น จำเป็นต้องมีสิ่งที่เรียกว่า MoveBase หรือสรุปง่ายๆ ก็คือส่วนในการวางแผนเส้นทางของหุ่นยนต์ MoveBase เป็นเครื่องมือที่ช่วยให้หุ่นยนต์สามารถควบคุมการเคลื่อนที่ของตัวเองไปยังเป้าหมายที่กำหนดได้โดยอัตโนมัติ ใช้งานร่วมกับระบบสั่งการการนำทาง (Navigation System) ของ ROS เพื่อทำการวางแผนเส้นทางและการเคลื่อนที่ของหุ่นยนต์ โดยอาศัยข้อมูล Sensor หรือแผนที่ในการคำนวณเส้นทางที่เหมาะสม และควบคุมการเคลื่อนที่ของหุ่นยนต์ให้เคลื่อนที่ตามเส้นทางที่วางแผนไว้ การใช้งาน MoveBase จะต้องกำหนดเป้าหมาย (Goal) ที่หุ่นยนต์ต้องการเคลื่อนที่ไปถึง ซึ่งคือพิกัดต่าง ๆ (Coordinates) หลังจากนั้น MoveBase จะทำการวางแผนเส้นทางและควบคุมการเคลื่อนที่ของหุ่นยนต์ให้เคลื่อนที่ไปยังเป้าหมายที่กำหนดไว้ ดังรูปที่5.13



รูปที่ 5.13 โครงสร้างการทำงานของหุ่นยนต์

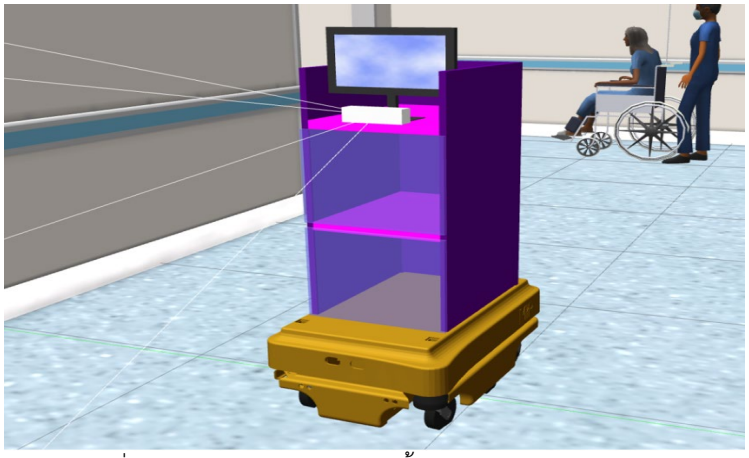
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ

ในบทนี้จะกล่าวถึงผลการดำเนินงานข้อจำกัดและข้อเสนอแนะจากการจัดทำโครงงานการจำลองหุ่นยนต์รับ-ส่งสิ่งของในโรงพยาบาลแผนกผู้ติดเชื้อโควิด-19 โดยระบบสามารถทำงานได้ดังนี้

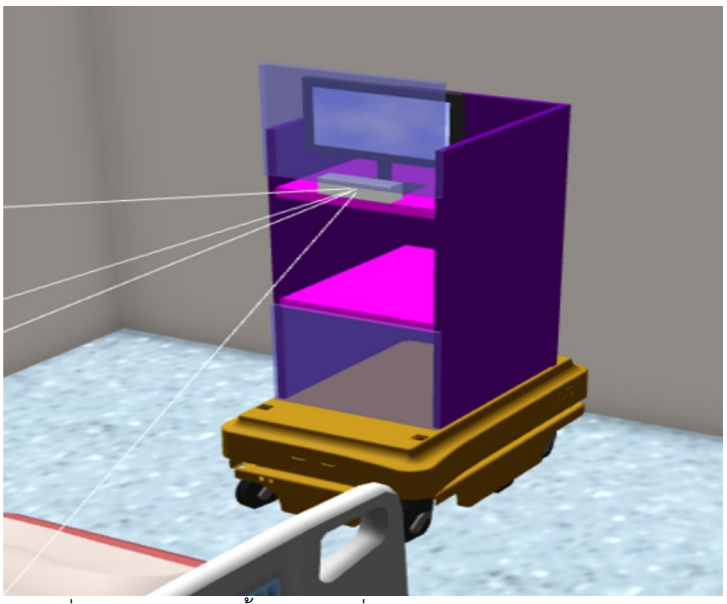
6.1 ผลการดำเนินงาน

6.1.1 การใช้งานหุ่นยนต์ของหมอและพยาบาล

1) หมอและพยาบาลสามารถสั่งงานหุ่นยนต์ให้ไปส่งของเช่น น้ำ ยา อาหาร ให้กับผู้ป่วยได้ แทนการใช้งานบุคลากรทางการแพทย์ โดยสามารถสั่งงานได้ผ่านเว็ปแอปพลิเคชัน โดยหุ่นยนต์จะทำการวางแผนโดยอัตโนมัติในการเคลื่อนที่ไปยังจุดมุ่งหมาย โดยหุ่นยนต์มีการป้องกันความผิดพลาดในการป้องกันสิ่งของที่ส่ง หรือป้องกันการหยิบสิ่งของที่มาส่งผิด ด้วยการเปิดกระจกชั้นวางของโดยอัตโนมัติ เมื่อผู้ป่วยต้องการรับของ ผลลัพธ์ในการทดสอบดังรูปที่ 6.1 และ 6.2

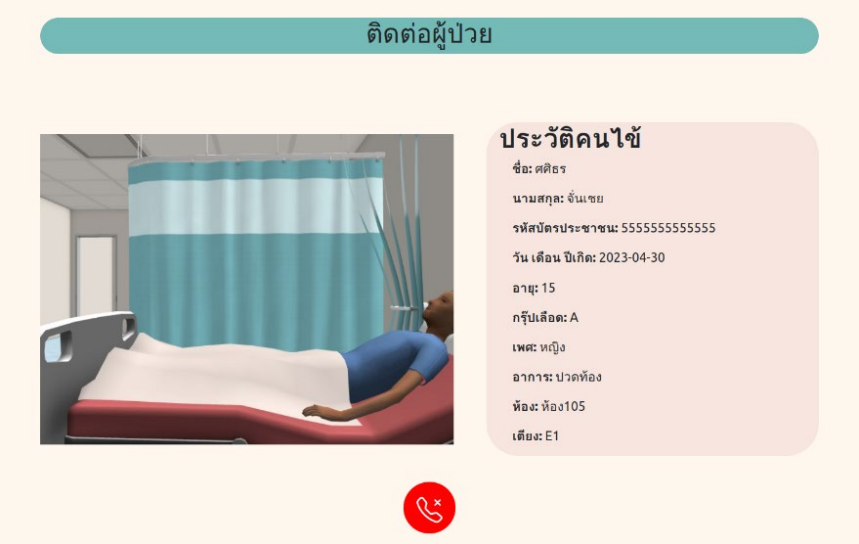


รูปที่ 6.1 หุ่นยนต์ขณะปิดกระจกชั้นวางของ



รูปที่ 6.2 หุ่นยนต์เปิดชั้นวางของเพื่อให้ผู้รับหยิบของ

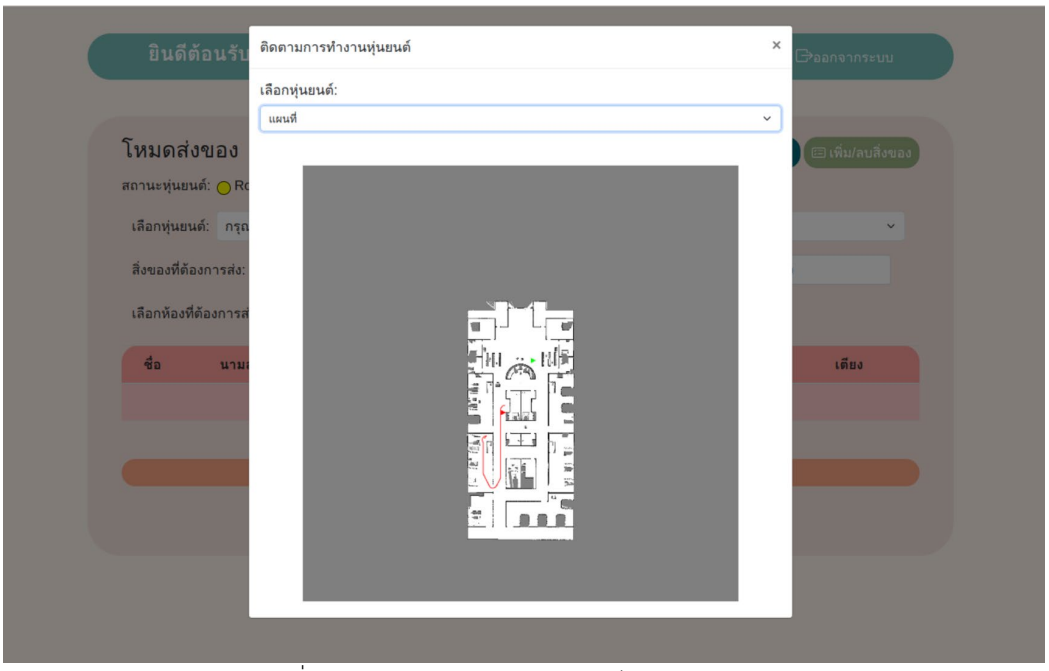
2) หมอและพยาบาลสามารถติดต่อดูอาการผู้ป่วยได้ผ่านหุ่นยนต์ โดยสามารถสั่งงานได้ผ่านเว็ปแอปพลิเคชัน ผลลัพธ์ในการทดสอบดังรูปที่ 6.3



รูปที่ 6.3 การติดต่อผู้ป่วยของหมอและพยาบาล

3) หมอและพยาบาลสามารถส่งหุ่นยนต์ไปยังที่ต่าง ๆ ได้ตามที่ต้องการ โดยสามารถสั่งงานได้ผ่านเว็ปแอป-พลิเคชัน

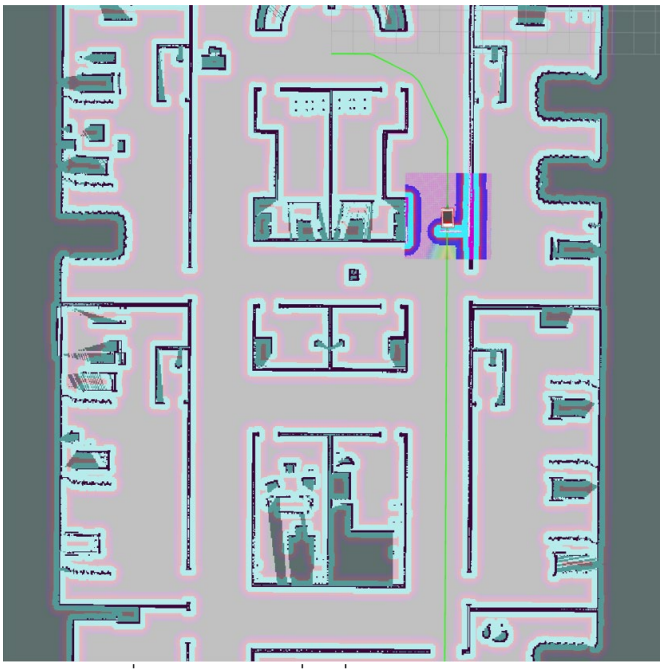
4) หมอและพยาบาล สามารถติดตามตำแหน่งของหุ่นยนต์ได้ ว่าหุ่นยนต์อยู่ส่วนไหนของโรงพยาบาล รวมถึงเส้นทางที่หุ่นยนต์วางแผนและกำลังทำการเคลื่อนที่ไปยังจุดหมาย ผลลัพธ์ดังรูปที่ 6.4



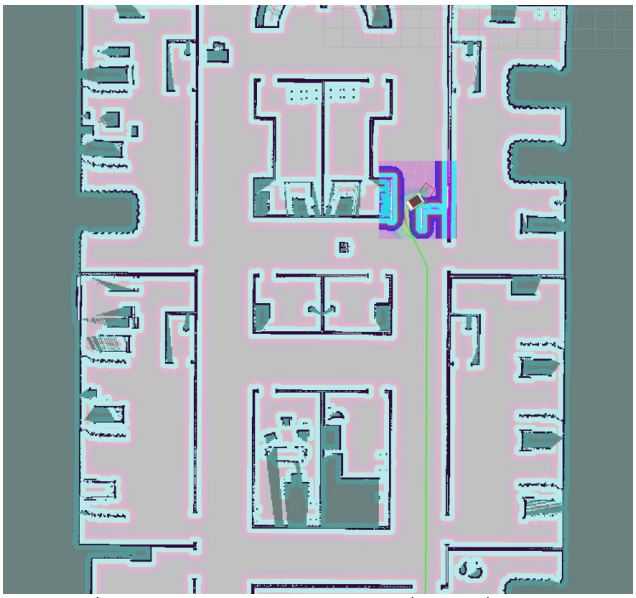
รูปที่ 6.4 การติดตามตำแหน่งหุ่นยนต์

6.1.2 การทำงานของหุ่นยนต์

หุ่นยนต์สามารถเคลื่อนที่ในสภาพแวดล้อมโรงพยาบาลได้ ด้วยการเคลื่อนที่ในรูปแบบอัติโนมัติ โดยหุ่นยนต์จะทำการวางแผนจากต้นทางไปยังปลายทาง และเคลื่อนที่ตามเส้นทางที่ได้สร้างขึ้น ในกรณีที่มีสิ่งกีดขวาง ขัดขวางเส้นทางอยู่ หุ่นยนต์จะทำการรอเป็นเวลา 15 วินาที ถ้าสิ่งกีดขวางหลบออกจากส้นทางหุ่นยนต์จะทำการเคลื่อนที่ต่อในเส้นทางเดิม แต่ในกรณีที่รอจนเวลาครบ หุ่นยนต์จะทำการคิดเส้นทางใหม่โดยอัตมัติอีกครั้งและเคลื่อนที่ตามเส้นทางที่ได้สร้างขึ้น โดยผลลัพธ์การทดลองดังรูปที่ 6.5 และ 6.6



รูปที่ 6.5 เหตุการณ์เมื่อมีสิ่งกีดขวาง ขวางเส้นทาง



รูปที่ 6.6 การวางแผนเส้นทางใหม่ เพื่อหลบสิ่งกีดขวาง

6.2 ข้อจำกัด

- ในแอปพลิเคชันยังไม่ถูกพัฒนาให้ผู้ใช้งานสามารถเข้าควบคุมการตั้งค่าความเร็วของหุ่นยนต์ได้ผ่านหน้า UI ของระบบติดต่อกับผู้ใช้งาน
- ในปัจจุบันสามารถสั่งงานในหมวดส่งของ ได้มากสุดรอบละ 2 งาน โดยเมื่อสั่งงานแล้ว จะไม่สามารถสั่งงานต่อได้ในทันทีในรูปแบบคิว จะสั่งงานอีกครั้งได้ก็ต่อเมื่อหุ่นยนต์ทำงานจนเสร็จและอยู่ในสถานะพร้อมใช้งาน

6.3 ข้อเสนอแนะ

- สร้าง UI ให้ผู้ใช้งาน สามารถควบคุมการตั้งค่าความไวได้ผ่าน UI
- พัฒนาให้เปลี่ยนเป็นรูปแบบคำสั่งที่ต่อคิวการทำงานได้เรื่อย ๆ

บรรณานุกรม

[1] มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี,“Tech2Biz,”2016.Available:https://www.tech2biz.net/content/3020-CARVER

[2] ร. ส. ณรงค์เดช กีรติพรานนท์, “ระบบเคลื่อนที่อัตโนมัติสำหรับหุ่นยนต์ประชาสัมพันธ์,” *Journal of Energy and Environment Technology,* 2018.

[3] C. Schultz, “rosbridge_suite,” Ros.org, 2022. Available:http://wiki.ros.org/rosbridge_suite.

[4] C. Fritz, “roslibjs,” ROS.org, 2022. Available: http://wiki.ros.org/roslibjs.

[5] D. Chikurtev, “Mobile Robot Simulation and Navigation in ROS and Gazebo,” 2020.

[6] F. U. P. A. d. S. L. C. R. C. Pedro Medeiros de Assis Brasi, “A Study on Global Path Planners Algorithms for the,” 2020.

[7] G. P. H. Mary B. Alatise, “A Review on Challenges of Autonomous Mobile Robot and Sensor Fusion Methods”.

[8] K. S. M. S. Hesham Ibrahim Mohamed Ahmed Omara, “Indoor mapping using kinect and ROS,” 2015.

[9] MatSadowski, “web_video_server,” ROS.org, 2019. Available:http://wiki.ros.org/web_video_server.

[10] R. Toris, “ros2djs,” ROS.org, 2015. Available: http://wiki.ros.org/ros2djs.

[11] TullyFoote, “Robot Operating System,” ROS.org, 2022. Available: http://wiki.ros.org/.

[12] WilliamWoodall, “rviz,” ROS.org, 2018. Available: http://wiki.ros.org/rviz.

[13] “A* Search Algorithm,” geeksforgeeks,2022.Available:https://www.geeksforgeeks.org/a-search-algorithm/.

[14] “MySQL,” ORACLE, 2023. Available: https://www.mysql.com/.

[15] “[AMCL] Robot Localization,”2018.Available:https://myrobod.wordpress.com/2018/06/ 25/amcl-robot-localization/.

[16] “MiR,” Mobile Industrial Robots, 2023. Available: https://www.mobile-industrialrobots .com/solutions/robots/mir100/.

[17] "Open Source Robotics Foundation," Gazebo, 2019. Available: http://gazebosim.org/..

[18] "Dijkstra's Shortest Path Algorithm | Greedy Algo-7," geeksforgeeks, 2023. Available: https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/.

[19] "eventemitter2," NPM, 2022. Available: https://www.npmjs.com/package/eventemitter2.

[20] "EaseUs," createjs, 2018. Available: https://createjs.com/easeljs.

ภาคผนวก ก การติดตั้งโปรแกรมในการทำงาน

ก.1 การติดตั้ง ROS Noetic

ก.1.1 การตั้งค่า source. List

ตั้งค่าคอมพิวเตอร์ให้ยอมรับซอฟต์แวร์จาก package.ros.org โดยใช้คำสั่งผ่านTerminal

- sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'

ก.1.2 การตั้งค่าคีย์

ตั้งค่าคีย์ด้วยคำสั่งผ่านTerminal

- sudo apt install curl # if you haven't already installed curl
- curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add -

ก.1.3 การติดตั้ง ROS Noetic

ก่อนอื่นตรวจสอบให้แน่ใจว่าแพคเกจของระบบปฏิบัติการและดาวน์โหลดรายการแพคเกจที่เป็นรายการอัปเดตล่าสุด โดยใช้คำสั่ง

- sudo apt update

ติดตั้งแบบ **Desktop-Full โดยคำสั่ง**

- sudo apt install ros-noetic-desktop-full

ก.1.4 การตั้งค่า Environment

- source /opt/ros/noetic/setup.bash

ก.2 การติดตั้ง Web Video Server

ก.2.1 การติดตั้ง Web Video Server

ติดตั้ง Web Video Server โดยใช้คำสั่ง

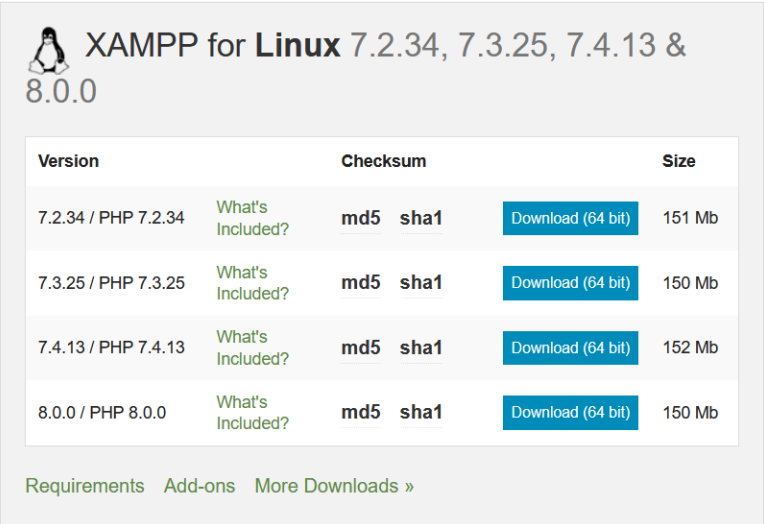
- cd ~/catkin/src
- git clone https://github.com/sfalexrog/async_web_server_cpp.git

- cd async_web_server
- git checkout noetic-devel
- cd ~/catkin/src
- git clone https://github.com/RobotWebTools/web_video_server.git
- catkin build

ก.3 การติดตั้ง XAMPP

ก.3.1 การดาวน์โหลด XAMPP

ดาวน์โหลด XAMPP ที่ https://www.apachefriends.org/download.html เลือกในส่วน XAMPP for Linux ดังรูปผนวกที่ ก.1



รูปผนวกที่ ก.2 รูปแสดงดาวน์โหลด XAMPP

เมื่อ Download มาแล้วให้เปิด Terminal ไปยัง Folder ที่เรา Download ไฟล์เก็บไว้ (ตัวอย่างเก็บไว้ที่ Downloads) โดยใช้คำสั่ง

- cd Downloads
- sudo chmod 755 xampp-linux-x64-7.2.34-0-installer.run

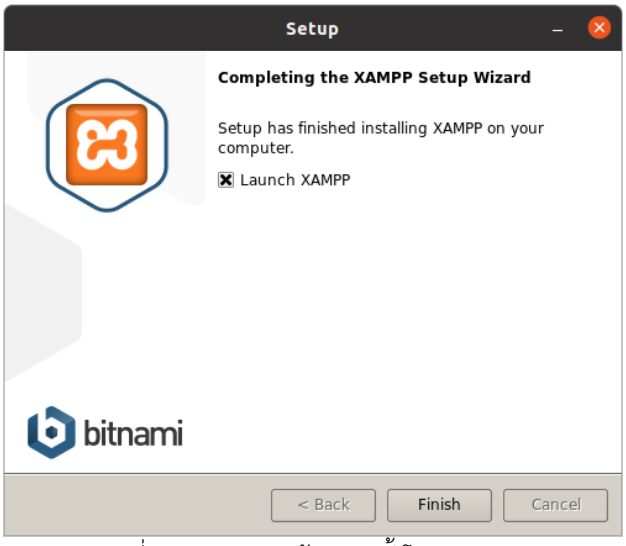
ใช้คำสั่งด้านล่างเพื่อติดตั้งโปรแกรม

- sudo ./xampp-linux-7.2.34-0-installer.run

จะแสดงหน้าจอติดตั้งโปรแกรม คลิก Next ไปเรื่อย ๆ จนเสร็จติดตั้งเสร็จดังรูปผนวกที่ ก.2

รูปผนวกที่ ก.3 รูปแสดงหน้าจอติดตั้งโปรแกรม XAMPP

คลิกปุ่ม Finish จะแสดงหน้าจอดังภาพ ให้กดปิดหน้าจอนี้ไปได้เลย ดังรูปผนวกที่ ก.3



รูปผนวกที่ ก.4 รูปแสดงหน้าจอติดตั้งโปรแกรม XAMPP

ก.4 การติดตั้ง Flask

ก.4.1 การตรวจสอบการUpdateก่อนติดตั้ง

ติดตั้งตรวจสอบให้แน่ใจว่าระบบได้รับการอัพเดตตามที่แสดง โดยใช้คำสั่งในTerminal

- sudo apt update -y

ก.4.2 การติดตั้ง pip3 และเครื่องมือ Python อื่น ๆ

- sudo apt install build-essential python3-pip libffi-dev python3-dev python3-setuptools libssl-dev

ก.4.3 การติดตั้งEnvironment

- sudo apt install python3-venv

ก.4.4 การสร้างไดเร็กทอรี flask

- mkdir flask_dir && cd flask_dir

ก.4.5 การสร้าง Environment โดยใช้ Python

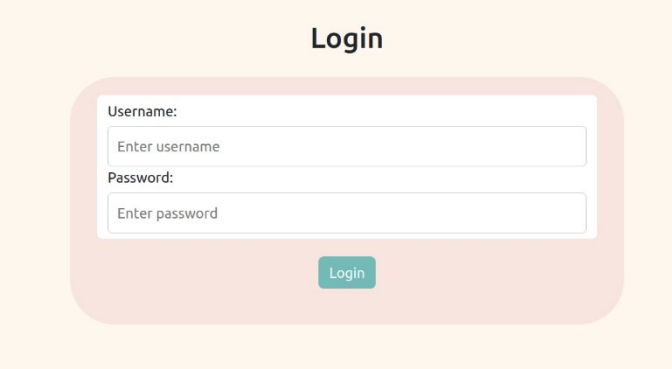
- python3 -m venv venv

ก.4.6 การติดตั้ง เว็บเฟรมเวิร์ก flask **โดยใช้** pip

- pip3 install flask

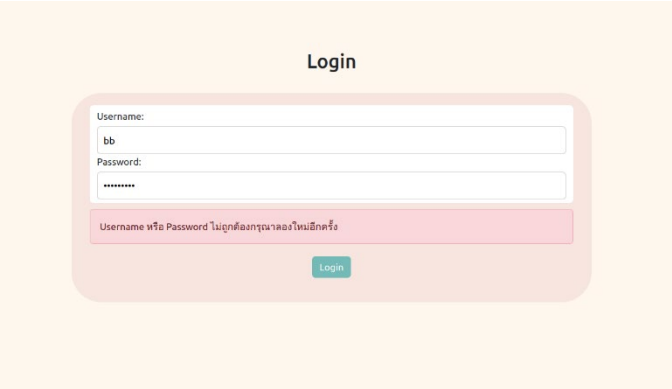
ภาคผนวก ข คู่มือการใช้งาน

หน้าเข้าสู่ระบบ โดยต้องใช้ User ที่มีอยู่ในระบบ โดยมีเพียงผู้ดูแลระบบเท่านั้นที่สร้าง User เพิ่มได้ ดังรูปผนวกที่ ข.1



รูปผนวกที่ ข.1 รูปแสดงหน้าเข้าสู่ระบบ

หากใส่ Username หรือ Password ผิด หรือกรณีที่ Username และ Password ที่ใส่ไม่มีอยู่ในระบบ จะมีการแจ้งเตือนข้อผิดพลาดขึ้นดังรูปผนวกที่ ข.2



รูปผนวกที่ ข.2 รูปแสดงหน้าเข้าสู่ระบบ มีการแจ้งเตือนข้อผิดพลาดขึ้น

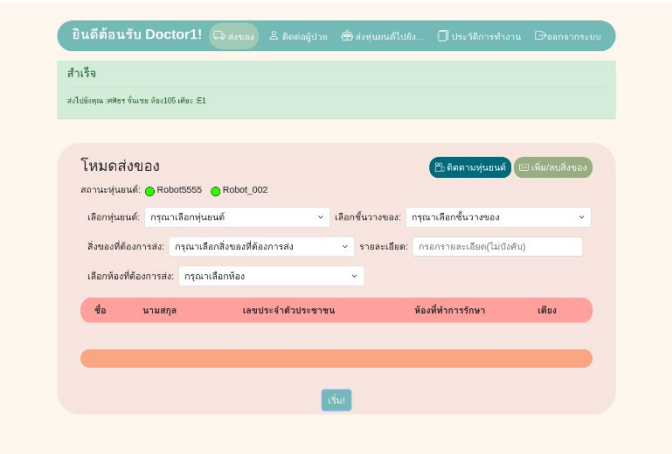
โดยเมื่อเข้าสู่ระบบ หากUsername มีสถานะเป็นหมอ ก็จะนำไปสู่การเข้าใช้งานในหมวดของหมอ โดยโหมดส่งของต้องทำการเลือกให้ครบทุกช่อง ยกเว้นช่องรายละเอียด ที่ไม่บังคับในการกรอกรายละเอียดเพิ่มเติม โดยจะไม่สามารถสั่งงานหุ่นยนต์ที่ไม่พร้อมใช้งานได้ โดยหุ่นยนต์มีทั้งหมด3สถานะ คือ สีเขียวพร้อมใช้งาน สีเหลืองกำลังทำงานอยู่ยังไม่พร้อมใช้งานในตอนนี้ สีแดงหุ่นยนต์ไม่สามารถสั่งการได้ดังรูปผนวกที่ ข.3

รูปผนวกที่ ข.3 รูปแสดงหมวดการทำงานของหมอโหมดส่งของ

หากเพิ่มการทำงานสำเร็จ จะแสดงในลักษณะนี้โดยแสดงคิวการทำงานที่เพิ่มไป และสามารถกดลบได้ที่ปุ่มลบดังรูปผนวกที่ ข.4

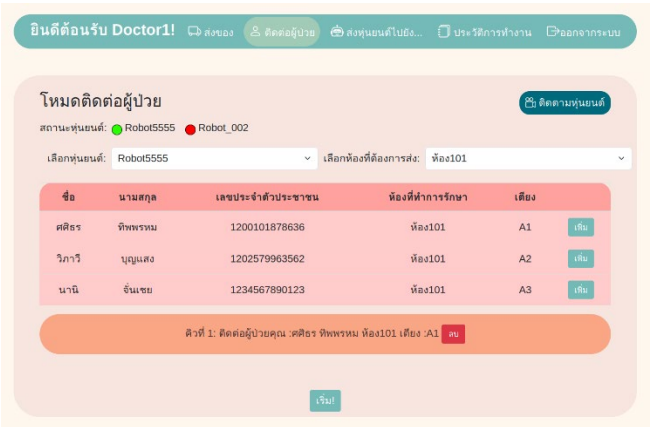
รูปผนวกที่ ข.4 รูปแสดงโหมดส่งของ แสดงคิวการทำงานที่เพิ่มไป และสามารถกดลบได้

เมื่อกดเริ่มการทำงาน จะแสดง Feedback ให้เห็นว่าสั่งสำเร็จแล้วดังรูปผนวกที่ ข.5



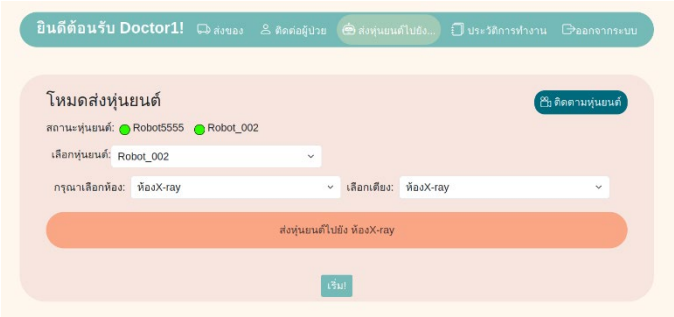
รูปผนวกที่ ข.5 รูปแสดง Feedback ให้เห็นว่าสั่งสำเร็จแล้ว

ในฟังก์ชันติดต่อผู้ป่วย ผู้ใช้ต้องเลือกการทำงานให้ครบก่อน จึงจะเริ่มการทำงานติดต่อผู้ป่วยได้ ดังรูปผนวกที่ ข.6



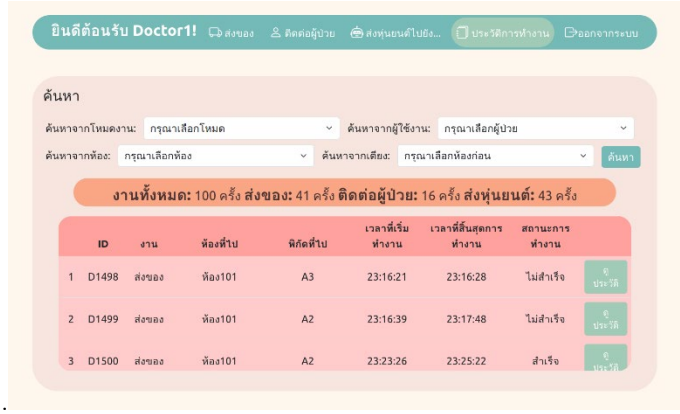
รูปผนวกที่ ข.6 รูปแสดงโหมดติดต่อผู้ป่วย

ฟังก์ชันส่งหุ่นยนต์ ผู้ใช้จะต้องเลือกการทำงานให้ครบ จึงจะสามารถ เริ่มการทำงานได้ ดังรูปผนวกที่ ข.7



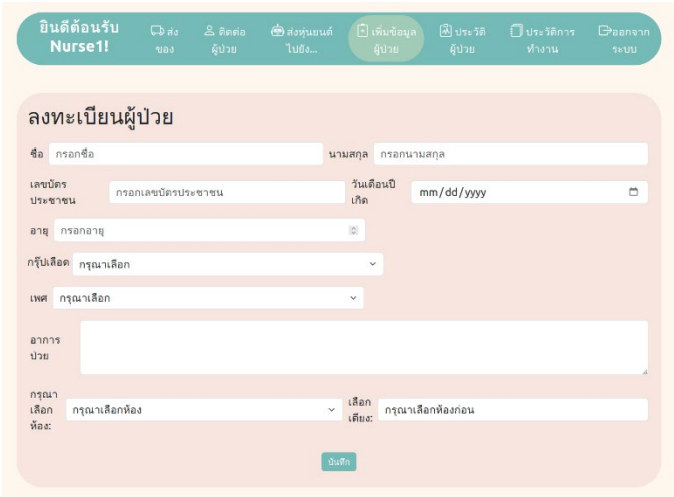
รูปผนวกที่ ข.7 รูปแสดงโหมดส่งหุ่นยนต์

ฟังก์ชันประวัติการทำงาน ผู้ใช้สามารถค้นหาประวัติการทำงานของหุ่นยนต์ได้ ดังรูปผนวกที่ ข.8



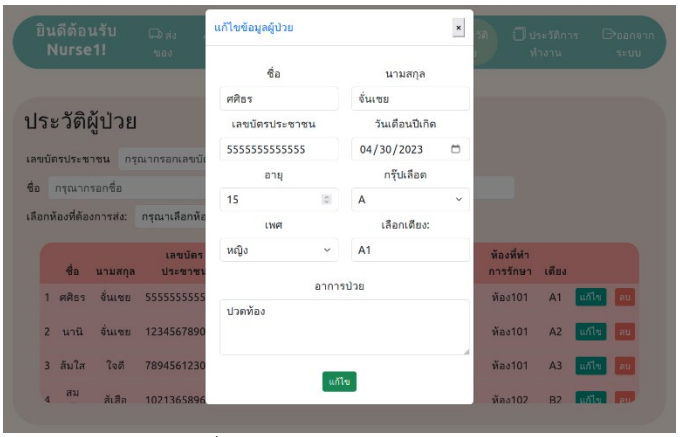
รูปผนวกที่ ข.8 รูปแสดงฟังก์ชันประวัติการทำงาน

หากเข้าสู่ระบบด้วย Username ที่มีสถานะเป็นพยาบาล ก็จะนำไปสู่การเข้าใช้งานในหมวดของพยาบาล โดยพยาบาลจะมีการทำงาน2ฟังก์ชันที่เพิ่มจากหมอ คือ เพิ่มข้อมูลผู้ป่วย และ ลบ/แก้ไข ประวัติผู้ป่วย ในฟังก์ชันการเพิ่มข้อมูลผู้ป่วย ผู้ใช้จำเป็นต้องกรอกข้อมูลให้ครบทุกช่องเพื่อเพิ่มผู้ป่วยไปยังตำแหน่งห้องต่าง ๆ ที่มี ดังรูปผนวกที่ ข.9



รูปผนวกที่ ข.9 รูปแสดงฟังก์ชันการเพิ่มข้อมูลผู้ป่วย

ฟังก์ชันประวัติผู้ป่วย ผู้ใช้สามารถค้นหา ลบ และแก้ไข ข้อมูลผู้ป่วยได้ ดังรูปผนวกที่ ข.10



รูปผนวกที่ ข.10 รูปแสดงฟังก์ชันประวัติผู้ป่วย

หากเข้าสู่ระบบด้วย Username ที่มีสถานะเป็นผู้ดูแลระบบ ก็จะนำไปสู่การเข้าใช้งานในหมวดของผู้ดูแลระบบ โดยผู้ดูแลระบบ จะมีฟังก์ชันการทำงานทั้งหมด 3 ฟังก์ชัน

1.) ฟังก์ชันการตั้งค่าผู้ใช้งาน สามารถค้นหา-เพิ่ม-ลบ-แก้ไขผู้ใช้ที่จะเข้าสู่แอปพลิเคชันได้ ดังรูปผนวกที่ ข.11

รูปผนวกที่ ข.11 รูปแสดงฟังก์ชันการตั้งค่าผู้ใช้งาน

2.) ฟังก์ชันการตั้งค่าพิกัดปลายทางสามารถค้นหา-เพิ่ม-ลบ-แก้ไขพิกัดต่าง ๆ ที่หุ่นยนต์ใช้ในการเคลื่อนที่ไปยังตำแหน่งนั้นๆ ดังรูปผนวกที่ ข.12

รูปผนวกที่ ข.12 รูปแสดงฟังก์ชันการตั้งค่าพิกัดปลายทาง

3.) ฟังก์ชันการตั้งค่าหุ่นยนต์ สามารถเปิด-ปิดหุ่นยนต์ได้ และตั้งค่าแก้ไข ชื่อหุ่นยนต์ และตำแหน่งเริ่มต้นของหุ่นยนต์ ดังรูปผนวกที่ ข.13

รูปผนวกที่ ข.13 รูปแสดงฟังก์ชันการตั้งค่าหุ่นยนต์